

LAB - ASSIGNMENT – 11.2

NAME: CH VISHWANATH RAO

BATCH: 02

HT NO: 2303A51095

Task Description #1 – Stack Implementation

Task: Use AI to generate a Stack class with push, pop, peek, and is_empty methods.

Sample Input Code:

```
class Stack:
```

```
pass
```

Expected Output:

- A functional stack implementation with all required methods and docstrings.

CODE:

```
# generate a optimized python program Stack class with push, pop, peek, and is_empty

class Stack:
    """
    A stack implementation using a list.
    """
    def __init__(self):
        """Initialize an empty stack"""
        self._items = []

    def push(self, item):
        """
        Push an item onto the stack.
        """
        self._items.append(item)

    def pop(self):
        """
        Pop an item from the stack.
        """
        if self.is_empty():
            raise IndexError("pop from empty stack")
        return self._items.pop()

    def peek(self):
        """
        Peek at the top item on the stack.
        """
        if self.is_empty():
            raise IndexError("peek from empty stack")
        return self._items[-1]

    def is_empty(self):
        """
        Check if the stack is empty.
        """
        return len(self._items) == 0
```

```
# Example usage:
if __name__ == "__main__":
    s = Stack()
    s.push(11)
    s.push(22)
    s.push(33)
    print(s.peek())
    print(s.pop())
    print(s.pop())
    print(s.pop())
    print(s.is_empty())
```

OUTPUT:

```
PS C:\Users\chvis\OneDrive\Desktop\ai\lab_11> & C:/Python314/python.exe c:/Users/chvis/OneDrive/Desktop/ai/lab_11/dem.py
33
33
22
11
True
PS C:\Users\chvis\OneDrive\Desktop\ai\lab_11>
```

Task Description #2 – Queue Implementation

Task: Use AI to implement a Queue using Python lists.

Sample Input Code:

class Queue:

pass

Expected Output:

- FIFO-based queue class with enqueue, dequeue, peek, and size methods.

CODE:

```
GENERATE A PYTHON PROGRAM THAT IMPLEMENTS A QUEUE FIFO-based queue class with enqueue, dequeue, peek, and size Methods.
class Queue:
    """
    A queue implementation using a list.
    """
    def __init__(self):
        """Initialize an empty queue"""
        self._items = []

    def enqueue(self, item):
        """
        Add an item to the end of the queue.
        """
        self._items.append(item)

    def dequeue(self):
        """
        Remove and return the item at the front of the queue.
        """
        if self.is_empty():
            raise IndexError("dequeue from empty queue")
        return self._items.pop(0)

    def peek(self):
        """
        Peek at the front item of the queue without removing it.
        """
        if self.is_empty():
            raise IndexError("peek from empty queue")
        return self._items[0]

    def size(self):
        """
        Return the number of items in the queue.
        """
        return len(self._items)

    def is_empty(self):
        """
        Check if the queue is empty.
        """
        return len(self._items) == 0
```

```

# Example usage:
if __name__ == "__main__":
    q = Queue()
    q.enqueue(10)
    q.enqueue(25)
    q.enqueue(35)
    print(q.peek())
    print(q.dequeue())
    print(q.dequeue())
    print(q.dequeue())
    print(q.is_empty())
    print(q.size())

```

OUTPUT:

```

PS C:\Users\chvis\OneDrive\Desktop\ai\lab_11> & C:/Python314/python.exe c:/Users/chvis/OneDrive/Desktop/ai/lab_11/dem.py
10
25
35
True
0
PS C:\Users\chvis\OneDrive\Desktop\ai\lab_11>

```

Task Description #3 – Linked List

Task: Use AI to generate a Singly Linked List with insert and display methods.

Sample Input Code:

class Node:

pass

class LinkedList:

pass

Expected Output:

- A working linked list implementation with clear method documentation.

CODE:

```

GENERATE A PYTHON PROGRAM generate a singly linked list with insert and display methods and a working linked list implementation with clear method documentation.
class Node:
    """
    A node in a singly linked list.
    """
    def __init__(self, data):
        """Initialize a node with data and a pointer to the next node."""
        self.data = data
        self.next = None

class SinglyLinkedList:
    """
    A singly linked list implementation.
    """
    def __init__(self):
        """Initialize an empty linked list."""
        self.head = None

    def insert(self, data):
        """
        Insert a new node with the given data at the end of the list.

        Parameters:
        data: The data to be stored in the new node.
        """
        new_node = Node(data)
        if self.head is None:
            self.head = new_node
            return
        last_node = self.head
        while last_node.next:
            last_node = last_node.next
        last_node.next = new_node

    def display(self):
        """
        Display the contents of the linked list.

        Returns:
        A string representation of the linked list.
        """
        nodes = []
        current_node = self.head
        while current_node:
            nodes.append(str(current_node.data))
            current_node = current_node.next
        return " -> ".join(nodes)

```

Task Description #4 – Binary Search Tree (BST)

Task: Use AI to create a BST with insert and in-order traversal methods.

Sample Input Code:

class BST:

pass

Expected Output:

- BST implementation with recursive insert and traversal methods

CODE:

```
#Generate a create a BST with insert and in-order traversal methods
class TreeNode:
    """
    A node in a binary search tree.
    """
    def __init__(self, data):
        """Initialize a tree node with data and pointers to left and right children."""
        self.data = data
        self.left = None
        self.right = None

class BinarySearchTree:
    """
    A binary search tree implementation.
    """
    def __init__(self):
        """Initialize an empty binary search tree."""
        self.root = None

    def insert(self, data):
        """
        Insert a new node with the given data into the binary search tree.

        Parameters:
        data: The data to be stored in the new node.
        """
        if self.root is None:
            self.root = TreeNode(data)
        else:
            self._insert_recursive(self.root, data)

    def _insert_recursive(self, node, data):
        """Helper method to insert a node recursively."""
        if data < node.data:
            if node.left is None:
                node.left = TreeNode(data)
            else:
                self._insert_recursive(node.left, data)
        else:
            if node.right is None:
                node.right = TreeNode(data)
            else:
                self._insert_recursive(node.right, data)

    def in_order_traversal(self):
        """
        Perform an in-order traversal of the binary search tree.

        Returns:
        A list of values in in-order traversal order.
        """
        return self._in_order_recursive(self.root)

    def _in_order_recursive(self, node):
        """Helper method to perform in-order traversal recursively."""
        result = []
        if node:
            result.extend(self._in_order_recursive(node.left))
            result.append(node.data)
            result.extend(self._in_order_recursive(node.right))
        return result
```

Task Description #5 – Hash Table

Task: Use AI to implement a hash table with basic insert, search, and delete methods.

Sample Input Code:

```
class HashTable:
```

```
pass
```

Expected Output:

- Collision handling using chaining, with well-commented methods.

CODE:

```
class HashTable:
    """
    A hash table implementation using separate chaining for collision resolution.
    """
    def __init__(self, size=10):
        """Initialize the hash table with a specified size."""
        self.size = size
        self.table = [[] for _ in range(size)]

    def _hash(self, key):
        """Generate a hash for the given key."""
        return hash(key) % self.size

    def insert(self, key, value):
        """
        Insert a key-value pair into the hash table.

        Parameters:
        key: The key to be inserted.
        value: The value associated with the key.
        """
        index = self._hash(key)
        for i, (k, v) in enumerate(self.table[index]):
            if k == key:
                self.table[index][i] = (key, value) # Update existing key
                return
            self.table[index].append((key, value)) # Insert new key-value pair

    def retrieve(self, key):
        """
        Retrieve the value associated with the given key.

        Parameters:
        key: The key to be retrieved.

        Returns:
        The value associated with the key, or None if the key is not found.
        """
        index = self._hash(key)
        for k, v in self.table[index]:
            if k == key:
                return v
        return None # Key not found

    def delete(self, key):
        """
        Delete the key-value pair associated with the given key from the hash table.

        Parameters:
        key: The key to be deleted.

        Returns:
        True if the key was found and deleted, False otherwise.
        """
        index = self._hash(key)
        for i, (k, v) in enumerate(self.table[index]):
            if k == key:
                del self.table[index][i] # Remove the key-value pair
                return True
        return False # Key not found
```

Task Description #6 – Graph Representation

Task: Use AI to implement a graph using an adjacency list.

Sample Input Code:

```
class Graph:
```

```
pass
```

Expected Output:

- Graph with methods to add vertices, add edges, and display connections.

CODE:

```
class Graph:
    def __init__(self):
        self.adjacency_list = {}

    def add_vertex(self, vertex):
        if vertex not in self.adjacency_list:
            self.adjacency_list[vertex] = []

    def add_edge(self, vertex1, vertex2):
        if vertex1 in self.adjacency_list and vertex2 in self.adjacency_list:
            self.adjacency_list[vertex1].append(vertex2)
            self.adjacency_list[vertex2].append(vertex1) # For undirected graph

    def display(self):
        result = []
        for vertex, edges in self.adjacency_list.items():
            result.append(f"{vertex}: {' '.join(edges)}")
        return "\n".join(result)
```

Task Description #7 – Priority Queue

Task: Use AI to implement a priority queue using Python's heapq module.

Sample Input Code:

```
class PriorityQueue:
```

pass

Expected Output:

- Implementation with enqueue (priority), dequeue (highest priority), and display methods.

CODE:

```
Generate a python program to implement a enqueue (priority), dequeue (highest priority) and display methods & to implement a priority queue using python's heapq module. give big code with print statements

import heapq
class PriorityQueue:
    def __init__(self):
        self.elements = []

    def enqueue(self, item, priority):
        heapq.heappush(self.elements, (priority, item))
        print(f"Enqueued: {item} with priority {priority}")

    def dequeue(self):
        if not self.elements:
            raise IndexError("Dequeue from an empty priority queue")
        priority, item = heapq.heappop(self.elements)
        print(f"Dequeued: {item} with priority {priority}")
        return item

    def display(self):
        print("Priority Queue Contents:")
        for priority, item in sorted(self.elements):
            print(f"Item: {item}, Priority: {priority}")

# Sample usage:
if __name__ == "__main__":
    pq = PriorityQueue()
    pq.enqueue("Task 1", priority=3)
    pq.enqueue("Task 2", priority=1)
    pq.enqueue("Task 3", priority=2)
    pq.enqueue("Task 4", priority=4)
    pq.display()
    pq.dequeue()
    pq.dequeue()
    pq.display()
```

OUTPUT:

```
PS C:\Users\chris\OneDrive\Desktop\ai\lab_11> & C:/Python314/python.exe c:/Users/chris/OneDrive/Desktop/ai/lab_11/dem.py
Enqueued: Task 1 with priority 3
Enqueued: Task 2 with priority 1
Enqueued: Task 3 with priority 2
Enqueued: Task 4 with priority 4
Priority Queue Contents:
Item: Task 2, Priority: 1
Item: Task 3, Priority: 2
Item: Task 1, Priority: 3
Item: Task 4, Priority: 4
Dequeued: Task 2 with priority 1
Dequeued: Task 3 with priority 2
Priority Queue Contents:
Item: Task 1, Priority: 3
Item: Task 4, Priority: 4
```

Task Description #8 – Deque

Task: Use AI to implement a double-ended queue using `collections.deque`.

Sample Input Code:

```
class DequeDS:
```

```
pass
```

Expected Output:

- Insert and remove from both ends with docstrings.

CODE:

```
from collections import deque

class DoubleEndedQueue:
    def __init__(self):
        self.deque = deque()

    def add_front(self, item):
        self.deque.appendleft(item)
        print(f"Item '{item}' added to the front of the deque.")

    def add_rear(self, item):
        self.deque.append(item)
        print(f"Item '{item}' added to the rear of the deque.")

    def remove_front(self):
        if not self.is_empty():
            item = self.deque.popleft()
            print(f"Item '{item}' removed from the front of the deque.")
            return item
        else:
            print("Deque is empty. Cannot remove item from front.")
            return None

    def remove_rear(self):
        if not self.is_empty():
            item = self.deque.pop()
            print(f"Item '{item}' removed from the rear of the deque.")
            return item
        else:
            print("Deque is empty. Cannot remove item from rear.")
            return None

    def peek_front(self):
        if not self.is_empty():
            item = self.deque[0]
            print(f"Item at front of deque: '{item}'")
            return item
        else:
            print("Deque is empty. Cannot peek front item.")
            return None

    def peek_rear(self):
        if not self.is_empty():
            item = self.deque[-1]
            print(f"Item at rear of deque: '{item}'")
            return item
        else:
            print("Deque is empty. Cannot peek rear item.")
            return None

    def is_empty(self):
        return len(self.deque) == 0

    def display(self):
        if not self.is_empty():
            print("Deque contents:")
            for item in self.deque:
                print(item)
        else:
            print("Deque is empty.")
```

```

# Example usage:
if __name__ == "__main__":
    deq = DoubleEndedQueue()
    deq.add_rear(1)
    deq.add_rear(2)
    deq.add_front(0)
    deq.display()
    deq.peek_front()
    deq.peek_rear()
    deq.remove_front()
    deq.remove_rear()
    deq.display()
    print("Is deque empty?", deq.is_empty())
    deq.remove_front()
    print("Is deque empty?", deq.is_empty())

```

OUTPUT:

```

PS C:\Users\chvis\OneDrive\Desktop\ai\lab_11> & C:/Python314/python.exe c:/Users/chvis/OneDrive/Desktop/ai/lab_11/dem.py
Item '1' added to the rear of the deque.
Item '2' added to the rear of the deque.
Item '0' added to the front of the deque.
Deque contents:
0
1
2
Item at front of deque: '0'
Item at rear of deque: '2'
Item '0' removed from the front of the deque.
Item '2' removed from the rear of the deque.
Deque contents:
1
Is deque empty? False
Item '1' removed from the front of the deque.
Is deque empty? True
PS C:\Users\chvis\OneDrive\Desktop\ai\lab_11>

```

Task Description #9 Real-Time Application Challenge – Choose the Right Data Structure

Scenario:

Your college wants to develop a Campus Resource Management System that handles:

1. Student Attendance Tracking – Daily log of students entering/exiting the campus.
2. Event Registration System – Manage participants in events with quick search and removal.
3. Library Book Borrowing – Keep track of available books and their due dates.
4. Bus Scheduling System – Maintain bus routes and stop connections.
5. Cafeteria Order Queue – Serve students in the order they arrive.

Student Task:

- For each feature, select the most appropriate data structure from

The list below:

- o Stack
- o Queue
- o Priority Queue

- o Linked List
- o Binary Search Tree (BST)
- o Graph
- o Hash Table
- o Deque
- Justify your choice in 2–3 sentences per feature.
- Implement one selected feature as a working Python program with AI-assisted code generation.

Expected Output:

- A table mapping feature → chosen data structure → justification.
- A functional Python program implementing the chosen feature with comments and docstrings.

CODE:

```
class AttendanceManagementSystem:
    def __init__(self):
        self.attendance_table = {}

    def mark_attendance(self, student_id, date):
        if student_id not in self.attendance_table:
            self.attendance_table[student_id] = set()
        self.attendance_table[student_id].add(date)
        print(f"Marked attendance for student {student_id} on {date}.")

    def check_attendance(self, student_id, date):
        if student_id in self.attendance_table and date in self.attendance_table[student_id]:
            print(f"Student {student_id} was present on {date}.")
            return True
        else:
            print(f"Student {student_id} was absent on {date}.")
            return False

    def display_attendance(self, student_id):
        if student_id in self.attendance_table:
            dates = sorted(self.attendance_table[student_id])
            print(f"Attendance for student {student_id}: {' '.join(dates)}")
        else:
            print(f"No attendance records found for student {student_id}.")

# Example usage:
if __name__ == "__main__":
    ams = AttendanceManagementSystem()
    ams.mark_attendance(1, "2021-09-01")
    ams.mark_attendance(1, "2021-09-02")
    ams.mark_attendance(2, "2021-09-01")
    ams.mark_attendance(2, "2021-09-02")
    ams.display_attendance(1)
    ams.display_attendance(2)
    ams.check_attendance(1, "2021-09-01")
    ams.check_attendance(1, "2021-09-02")
    ams.check_attendance(2, "2021-09-01")
```

OUTPUT:

```
PS C:\Users\chvis\OneDrive\Desktop\ai\lab_11> & C:/Python314/python.exe c:/Users/chvis/OneDrive/Desktop/ai/lab_11/dem.py
● Marked attendance for student 1 on 2021-09-01.
Marked attendance for student 1 on 2021-09-02.
Marked attendance for student 2 on 2021-09-01.
Marked attendance for student 2 on 2021-09-02.
Attendance for student 1: 2021-09-01, 2021-09-02
Attendance for student 2: 2021-09-01, 2021-09-02
Student 1 was present on 2021-09-01.
Student 1 was present on 2021-09-02.
Student 2 was present on 2021-09-01.
```

Task Description #10: Smart Hospital Management System – Data - Structure Selection

A hospital wants to develop a Smart Hospital Management System that handles:

1. Patient Check-In System – Patients are registered and treated in order of arrival.
2. Emergency Case Handling – Critical patients must be treated first.
3. Medical Records Storage – Fast retrieval of patient details using ID.
4. Doctor Appointment Scheduling – Appointments sorted by time.
5. Hospital Room Navigation – Represent connections between wards and rooms.

Student Task

- For each feature, select the most appropriate data structure from the list below:
 - o Stack
 - o Queue
 - o Priority Queue
 - o Linked List
 - o Binary Search Tree (BST)
 - o Graph
 - o Hash Table
 - o Deque
- Justify your choice in 2–3 sentences per feature.
- Implement one selected feature as a working Python program with AI-assisted code generation.

Expected Output:

- A table mapping feature → chosen data structure → justification.
- A functional Python program implementing the chosen feature with comments and docstrings.

CODE:

```
import heapq

class Appointment:
    """
    Represents a single patient appointment.
    """
    Attributes
    -----
    patient_name : str
        The name of the patient who booked the appointment.
    appointment_time : str
        A datetime representation of the time of the appointment.
        In a real system this might be a datetime object, here we keep it
        simple and use a string for demonstration purposes.
    """
    def __init__(self, patient_name, appointment_time):
        """
        A constructor that takes names and the supplied values
        """
        self.patient_name = patient_name
        self.appointment_time = appointment_time
    def __lt__(self, other):
        """
        Defines the less-than operator so that heapq can compare instances.
        """
        """
        The heapq module uses the < operator to order elements. We delegate
        comparison to the appointment_time attribute, which must itself be
        comparable (strings, numbers, etc.).
        """
        return self.appointment_time < other.appointment_time

class SectorAppointmentSchedulingSystem:
    """
    A very basic scheduling system that keeps upcoming appointments
    in a priority queue.
    """
    """
    The internal representation is a list named appointments, which is
    maintained as a heap via the heapq module. The heap invariant
    guarantees that the smallest element (earliest time) is always at
    index 0.
    """
    def __init__(self):
        """
        A constructor to create a heap list that will act as the heap
        """
        self.appointments = []
    def schedule_appointment(self, patient_name, appointment_time):
        """
        Schedule a new appointment.
        """
        Parameters
        -----
        patient_name : str
            Name of the patient for whom the appointment is being booked.
        appointment_time : datetime
            Time of the appointment. Should support comparison operators.
        """
        appointment = Appointment(patient_name, appointment_time)
        """
        Append the new appointment onto the heap; items will float up
        & sit in the correct position to maintain the heap invariant.
        """
        heapq.heappush(self.appointments, appointment)
        print(f"Scheduled appointment for {patient_name} at {appointment_time}.")
    def get_next_appointment(self):
        """
        Remove and return the next (earliest) appointment.
        """
        Returns
        -----
        Appointment or None
            The Appointment object representing the next booking, or None if
            there are no appointments scheduled.
        """
        if not self.appointments:
            print("No appointments scheduled.")
            return None
        # pop the smallest element from the heap
        next_appointment = heapq.heappop(self.appointments)
        print(f"Next appointment: {next_appointment.patient_name} at {next_appointment.appointment_time}.")
        return next_appointment
    def display_appointments(self):
        """
        Print all currently scheduled appointments in order.
        """
        """
        This method does not modify the heap; it simply sorts a shallow copy
        for display purposes so that the internal state is preserved.
        """
        if not self.appointments:
            print("No appointments scheduled.")
            return
        print("Scheduled Appointments:")
        # currently will use the Appointment.__lt__ method defined above
        for appointment in sorted(self.appointments):
            print(f"({appointment.patient_name} at {appointment.appointment_time})")
# Example usage:
if __name__ == "__main__":
    scheduling_system = SectorAppointmentSchedulingSystem()
    scheduling_system.schedule_appointment("mahesh", "2021-09-01 12:00")
    scheduling_system.schedule_appointment("Babu", "2021-09-01 09:00")
    scheduling_system.schedule_appointment("Chari", "2021-09-01 03:00")
    scheduling_system.display_appointments()
    scheduling_system.get_next_appointment()
    scheduling_system.display_appointments()
```

OUTPUT:

```
PS C:\Users\chvis\OneDrive\Desktop\ai\lab_11> & C:/Python314/python.exe c:/Users/chvis/OneDrive/Desktop/ai/lab_11/dem.py
• Scheduled appointment for mahesh at 2021-09-01 12:00.
Scheduled appointment for Babu at 2021-09-01 09:00.
Scheduled appointment for Chari at 2021-09-01 03:00.
Scheduled Appointments:
Chari at 2021-09-01 03:00
Chari at 2021-09-01 03:00
Babu at 2021-09-01 09:00
mahesh at 2021-09-01 12:00
Next appointment: Chari at 2021-09-01 03:00.
Scheduled Appointments:
Babu at 2021-09-01 09:00
○ mahesh at 2021-09-01 12:00
PS C:\Users\chvis\OneDrive\Desktop\ai\lab_11> 
```

Task Description #11: Smart City Traffic Control System

A city plans a Smart Traffic Management System that includes:

1. Traffic Signal Queue – Vehicles waiting at signals.
2. Emergency Vehicle Priority Handling – Ambulances and fire trucks prioritised.
3. Vehicle Registration Lookup – Instant access to vehicle details.
4. Road Network Mapping – Roads and intersections connected logically.
5. Parking Slot Availability – Track available and occupied slots.

Student Task

- For each feature, select the most appropriate data structure from

The list below:

- o Stack
- o Queue
- o Priority Queue
- o Linked List
- o Binary Search Tree (BST)
- o Graph
- o Hash Table
- o Deque

- Justify your choice in 2–3 sentences per feature.
- Implement one selected feature as a working Python program with AI-assisted code generation.

Expected Output:

- A table mapping feature → chosen data structure → justification.
- A functional Python program implementing the chosen feature with comments and docstrings.

CODE:

```
from typing import List, Tuple, Optional
from dataclasses import dataclass, field
from itertools import count
import heapq
from collections import deque

# Unique mapping of features to data structures with custom explanations
FEATURE_LIST: List[Tuple[str, str]] = [
    (
        "Signal Queue for Vehicles",
        "Queue Structure",
        "Cars waiting at lights are handled in arrival sequence. "
        "This structure ensures first-in-first-out and promotes equitable movement."
    ),
    (
        "Priority for Urgent Vehicles",
        "Priority Queue",
        "Critical responders like medics and firefighters get precedence over regular traffic. "
        "This queue prioritizes urgent items based on importance level."
    ),
    (
        "Lookup for Vehicle Info",
        "Hash Map",
        "Details are often queried using license plates. "
        "Hash maps provide fast average time for searches, additions, and removals."
    ),
    (
        "Mapping of Road Systems",
        "Graph Representation",
        "Crossroads and pathways form nodes and connections. "
        "Graphs enable pathfinding and connectivity analysis."
    ),
    (
        "Status of Parking Spaces",
        "Hash Map",
        "Space identifiers link to availability states. "
        "Hash maps facilitate rapid queries and modifications."
    ),
]
```

```
def print_feature_mapping_table() -> None:
    print("\nURBAN TRAFFIC MANAGEMENT - STRUCTURE MAPPING")
    print("-" * 120)
    print(f"{'feature':<36} {'Data Structure':<18} {'Explanation':<40}")
    print("-" * 120)
    for item, struct, desc in FEATURE_LIST:
        print(f"{'item':<36} {'(struct:<18)} {'(desc)':<40}")
    print("-" * 120)

@dataclass(order=True)
class UrgentVehicle:
    urgency_level: int
    order_num: int
    unit_id: str = field(compare=False)
    unit_category: str = field(compare=False)
    spot: str = field(compare=False)

class UrgentVehiclePrioritySystem:
    def __init__(self) -> None:
        self._priority_list: List[UrgentVehicle] = []
        self._seq_gen = count()

    @staticmethod
    def _priority_desc(urgency_level: int) -> str:
        return {1: "TOP", 2: "ELEVATED", 3: "MODERATE", 4: "BASIC"}.get(urgency_level, "UNDEFINED")

    def register_unit(self, unit_id: str, unit_category: str, spot: str, urgency_level: int) -> None:
        entry = UrgentVehicle(
            urgency_level=urgency_level,
            order_num=next(self._seq_gen),
            unit_id=unit_id,
            unit_category=unit_category,
            spot=spot,
        )
        heapq.heappush(self._priority_list, entry)
        print(f"Registered: {unit_category} ({unit_id}) at {spot} [{self._priority_desc(urgency_level)}]")

    def send_next(self) -> Optional[UrgentVehicle]:
        if not self._priority_list:
            print("No urgent units pending.")
            return None
        entry = heapq.heappop(self._priority_list)
        print(f"Sending: {entry.unit_category} ({entry.unit_id}) from {entry.spot} [{self._priority_desc(entry.urgency_level)}]")
        return entry

    def check_next(self) -> Optional[UrgentVehicle]:
        if not self._priority_list:
            print("No urgent units pending.")
            return None
        entry = self._priority_list[0]
        print(f"Upcoming: {entry.unit_category} ({entry.unit_id}) at {entry.spot} [{self._priority_desc(entry.urgency_level)}]")
        return entry

    def has_no_entries(self) -> bool:
        return len(self._priority_list) == 0

    def show_list(self) -> None:
        if not self._priority_list:
            print("List is clear.")
            return
        print("\nUrgent List (top priority first):")
        print("-" * 80)
        for entry in sorted(self._priority_list):
            print(
                f"{'ID':<12} {entry.unit_id:<18} {'Category':<18} {entry.unit_category:<12} "
                f"{'Spot':<18} {entry.spot:<18} {'Urgency':<18} {self._priority_desc(entry.urgency_level)}"
            )
        print("-" * 80)
```

OUTPUT:

```
PS C:\Users\chvis\OneDrive\Desktop\ai\all git pdfs\LAB_11.2> & c:/python314/python.exe "c:/Users/chvis/OneDrive/Desktop/ai/all git pdfs/LAB_11.2/DEM.py"

URBAN TRAFFIC MANAGEMENT - STRUCTURE MAPPING
=====
Feature                                Data Structure    Explanation
-----
Signal Queue for Vehicles              Queue Structure    Cars waiting at lights are handled in arrival sequence. This structure ensures first-in-first-out and promotes equitable movement.
Priority for Urgent Vehicles            Priority Queue      Critical responders like medics and firefighters get precedence over regular traffic. This queue prioritizes urgent items based on importance level.
Lookup for Vehicle Info                Hash Map           Details are often queried using license plates. Hash maps provide fast average time for searches, additions, and removals.
Mapping of Road Systems                Graph Representation Crossroads and pathways form nodes and connections. Graphs enable pathfinding and connectivity analysis.
Status of Parking Spaces               Hash Map           Space identifiers link to availability states. Hash maps facilitate rapid queries and modifications.
=====

Registering urgent units:
Registered: Medical Unit (MED-101) at Crossroad A [TOP]
Registered: Fire Unit (FIRE-007) at Crossroad C [ELEVATED]
Registered: Police Unit (POL-055) at Crossroad B [MODERATE]
Registered: Medical Unit (MED-212) at Crossroad D [TOP]

Urgent List (top priority first):
-----
ID: MED-101 Category: Medical Unit Spot: Crossroad A Urgency: TOP
ID: MED-212 Category: Medical Unit Spot: Crossroad D Urgency: TOP
ID: FIRE-007 Category: Fire Unit Spot: Crossroad C Urgency: ELEVATED
ID: POL-055 Category: Police Unit Spot: Crossroad B Urgency: MODERATE
-----
Upcoming: Medical Unit (MED-101) at Crossroad A [TOP]

Sending process:
Sending: Medical Unit (MED-101) from Crossroad A [TOP]
Sending: Medical Unit (MED-212) from Crossroad D [TOP]
Sending: Fire Unit (FIRE-007) from Crossroad C [ELEVATED]
Sending: Police Unit (POL-055) from Crossroad B [MODERATE]
List is clear.
```

Task Description #12: Smart E-Commerce Platform – Data Structure Challenge

An e-commerce company wants to build a Smart Online Shopping System with:

1. Shopping Cart Management – Add and remove products dynamically.
2. Order Processing System – Orders processed in the order they are placed.
3. Top-Selling Products Tracker – Products ranked by sales count.
4. Product Search Engine – Fast lookup of products using product ID.
5. Delivery Route Planning – Connect warehouses and delivery locations.

Student Task

- For each feature, select the most appropriate data structure from the list below:

- o Stack
- o Queue
- o Priority Queue
- o Linked List
- o Binary Search Tree (BST)
- o Graph
- o Hash Table
- o Deque

- Justify your choice in 2–3 sentences per feature.
- Implement one selected feature as a working Python program with AI-assisted code generation.

Expected Output:

- A table mapping feature → chosen data structure → justification.
- A functional Python program implementing the chosen feature with comments and docstrings.

CODE

```
Smart E-commerce Platform | Data Structure Challenge
An e-commerce company wants to build a Smart Online Shopping System
with:
1. Shopping Cart Management | Add and remove products dynamically.
2. Order Processing System | Orders processed in the order they are placed.
3. Top-Selling Products Tracker | Products ranked by sales count.
4. Product Search Engine | Fast lookup of products using product ID.
5. Delivery Route Planning | Connect warehouses and delivery locations.
...

---
FEATURE MAPPING TABLE:
=====
1. Shopping Cart Management → Deque
   Justification: Allows adding/removing products from both ends efficiently.
   Supports undo (pop from end) and redo operations commonly needed in shopping carts.

2. Order Processing System → Queue
   Justification: Orders must be processed in FIFO order based on arrival time.
   Queue ensures fair processing and maintains chronological order for order fulfillment.

3. Top-Selling Products Tracker → Priority Queue
   Justification: Products are ranked by sales count with highest-selling items prioritized.
   Priority queue automatically maintains sorted order by sales frequency for quick access to top products.

4. Product Search Engine → Hash Table
   Justification: Provides O(1) average lookup time for products using product ID as key.
   Enables fast retrieval of product details like price, stock, and specifications.

5. Delivery Route Planning → Graph
   Justification: Represents warehouses and delivery locations as vertices with roads as edges.
   Graph enables pathfinding algorithms to optimize delivery routes and reduce shipping time.
...

:
:
```

```

from collections import deque
import heapq

class Item:
    def __init__(self, item_id, title, cost, amount):
        self.item_id = item_id
        self.title = title
        self.cost = cost
        self.amount = amount

    def __repr__(self):
        return f"Item({self.item_id}, {self.title}, ${self.cost}, Qty: {self.amount})"

class Cart:
    def __init__(self):
        self.items = deque()

    def add_item(self, item):
        self.items.append(item)
        print(f"✅ Added to cart: {item.title} - ${item.cost}")

    def add_item_front(self, item):
        self.items.appendleft(item)
        print(f"🌟 Priority added to cart: {item.title} - ${item.cost}")

    def remove_item(self):
        if not self.is_empty():
            item = self.items.pop()
            print(f"❌ Removed from cart: {item.title}")
            return item
        else:
            print("⚠️ Cart is empty. Cannot remove.")
            return None

    def remove_item_front(self):
        if not self.is_empty():
            item = self.items.popleft()
            print(f"❌ Removed from front: {item.title}")
            return item
        else:
            print("⚠️ Cart is empty. Cannot remove.")
            return None

    def get_total_cost(self):
        total = sum(item.cost * item.amount for item in self.items)
        return total

    def is_empty(self):
        return len(self.items) == 0

```



```

def display_cart(self):
    if self.is_empty():
        print("🛒 Your cart is empty.\n")
    else:
        print("\n🛒 CART CONTENTS:")
        print("-" * 70)
        total = 0
        for i, item in enumerate(self.items, 1):
            item_total = item.cost * item.amount
            total += item_total
            print(f"{i}. {item.title} | Cost: ${item.cost} | Qty: {item.amount} | Subtotal: ${item_total}")
        print("-" * 70)
        print(f"Total Cost: ${total}")
        print("-" * 70 + "\n")

# Example Usage
if __name__ == "__main__":
    print("-" * 70)
    print("🛒 SMART E-COMMERCE CART SYSTEM")
    print("-" * 70 + "\n")

    cart = Cart()

    # Create items
    i1 = Item(101, "Laptop", 999.99, 1)
    i2 = Item(102, "Wireless Mouse", 29.99, 2)
    i3 = Item(103, "USB-C Cable", 15.99, 3)
    i4 = Item(104, "Monitor", 299.99, 1)
    i5 = Item(105, "Keyboard", 79.99, 1)

    # Add items to cart
    print("🛒 Adding items to cart:\n")
    cart.add_item(i1)
    cart.add_item(i2)
    cart.add_item(i3)
    cart.add_item_front(i4) # Priority item
    cart.add_item(i5)

    # Display cart
    cart.display_cart()

    # Remove items (undo operation)
    print("🛒 Removing items (Undo operations):\n")
    cart.remove_item()
    cart.remove_item_front()

    # Display updated cart
    cart.display_cart()

    # Continue shopping
    print("🛒 Adding more items:\n")
    i6 = Item(106, "Headphones", 149.99, 1)
    cart.add_item(i6)

    # Final cart display
    cart.display_cart()

    # Summary
    print(f"🛒 Final Total: ${cart.get_total_cost():.2f}")

```

OUTPUT:

```
=====
SMART E-COMMERCE CART SYSTEM
=====

🛒 Adding items to cart:

✅ Added to cart: Laptop - $999.99
✅ Added to cart: Wireless Mouse - $29.99
✅ Added to cart: USB-C Cable - $15.99
🌟 Priority added to cart: Monitor - $299.99
✅ Added to cart: Keyboard - $79.99

📋 CART CONTENTS:
-----
1. Monitor | Cost: $299.99 | Qty: 1 | Subtotal: $299.99
2. Laptop | Cost: $999.99 | Qty: 1 | Subtotal: $999.99
3. Wireless Mouse | Cost: $29.99 | Qty: 2 | Subtotal: $59.98
4. USB-C Cable | Cost: $15.99 | Qty: 3 | Subtotal: $47.97
5. Keyboard | Cost: $79.99 | Qty: 1 | Subtotal: $79.99
-----
Total Cost: $1487.92
-----
Total Cost: $1487.92
-----
Total Cost: $1487.92
-----

🗑 Removing items (Undo operations):

❌ Removed from cart: Keyboard
❌ Removed from cart: Keyboard
❌ Removed from front: Monitor

📋 CART CONTENTS:
-----
1. Laptop | Cost: $999.99 | Qty: 1 | Subtotal: $999.99
2. Wireless Mouse | Cost: $29.99 | Qty: 2 | Subtotal: $59.98
3. USB-C Cable | Cost: $15.99 | Qty: 3 | Subtotal: $47.97
-----
1. Laptop | Cost: $999.99 | Qty: 1 | Subtotal: $999.99
2. Wireless Mouse | Cost: $29.99 | Qty: 2 | Subtotal: $59.98
3. USB-C Cable | Cost: $15.99 | Qty: 3 | Subtotal: $47.97
-----
2. Wireless Mouse | Cost: $29.99 | Qty: 2 | Subtotal: $59.98
3. USB-C Cable | Cost: $15.99 | Qty: 3 | Subtotal: $47.97
-----
Total Cost: $1107.94
-----
```

```
🛒 Adding more items:

✅ Added to cart: Headphones - $149.99
✅ Added to cart: Headphones - $149.99

📋 CART CONTENTS:
-----
1. Laptop | Cost: $999.99 | Qty: 1 | Subtotal: $999.99
2. Wireless Mouse | Cost: $29.99 | Qty: 2 | Subtotal: $59.98
3. USB-C Cable | Cost: $15.99 | Qty: 3 | Subtotal: $47.97
-----
1. Laptop | Cost: $999.99 | Qty: 1 | Subtotal: $999.99
2. Wireless Mouse | Cost: $29.99 | Qty: 2 | Subtotal: $59.98
3. USB-C Cable | Cost: $15.99 | Qty: 3 | Subtotal: $47.97
-----
1. Laptop | Cost: $999.99 | Qty: 1 | Subtotal: $999.99
2. Wireless Mouse | Cost: $29.99 | Qty: 2 | Subtotal: $59.98
3. USB-C Cable | Cost: $15.99 | Qty: 3 | Subtotal: $47.97
4. Headphones | Cost: $149.99 | Qty: 1 | Subtotal: $149.99
-----
Total Cost: $1257.93
-----
Total Cost: $1257.93
-----

✅ Final Total: $1257.93
```

Task #13: Smart Logistics & Warehouse Management

Scenario:

A logistics company wants to implement:

1. Shipment Processing
2. Urgent Shipment Priority
3. Product Lookup by Code
4. Warehouse Network Connectivity
5. Inventory Add/Remove Operations

Student Task

- For each feature, select the most appropriate data structure from the list below:
 - o Stack
 - o Queue
 - o Priority Queue
 - o Linked List
 - o Binary Search Tree (BST)
 - o Graph
 - o Hash Table
 - o Deque
- Justify your choice in 2–3 sentences per feature.
- Implement one selected feature as a working Python program with AI-assisted code generation.

Expected Output:

- A table mapping feature → chosen data structure → justification.
- A functional Python program implementing the chosen feature with comments and docstrings.

CODE:

```
***
Smart Logistics & Warehouse Management
Scenario:
A logistics company wants to implement:
1. Shipment Processing
2. Urgent Shipment Priority
3. Product Lookup by Code
4. Warehouse Network Connectivity
5. Inventory Add/Remove Operations
Student Task
• For each feature, select the most appropriate data structure from
the list below:
o Stack
o Queue
o Priority Queue
o Linked List
o Binary Search Tree (BST)
o Graph
o Hash Table
o Deque
• Justify your choice in 2-3 sentences per feature.
• Implement one selected feature as a working Python program with
AI-assisted code generation.
Expected Output:
• A table mapping feature > chosen data structure > justification..
• A functional Python program implementing the chosen feature
with comments and docstrings.
***

***
FEATURE MAPPING TABLE:
=====
1. Shipment Processing > Queue
Justification: Shipments must be processed in the order they arrive for fair distribution.
Queue's FIFO nature ensures shipments are handled chronologically without delays.

2. Urgent Shipment Priority > Priority Queue
Justification: Urgent shipments need expedited handling regardless of arrival time.
Priority queue allows sorting by urgency level to serve critical shipments first.

3. Product Lookup by Code > Hash Table
Justification: Provides O(1) average lookup time using product code as key.
Enables instant retrieval of product details, stock levels, and warehouse location.

4. Warehouse Network Connectivity > Graph
Justification: Warehouses are nodes and shipping routes are edges in the network.
Graph structure enables pathfinding and connectivity analysis between distribution centers.

5. Inventory Add/Remove Operation > Stack
Justification: Uses LIFO principle for inventory management (last added items are picked first).
Stack efficiently handles quick addition and removal of stock from warehouse shelves.
***
```

```
class Shipment:

    def __init__(self, shipment_id, destination, weight, status="Pending"):
        self.shipment_id = shipment_id
        self.destination = destination
        self.weight = weight
        self.status = status

    def __repr__(self):
        return f"Shipment({self.shipment_id}, {self.destination}, {self.weight}kg, {self.status})"

class ShipmentQueue:

    def __init__(self):
        self.queue = []
        self.processed = []

    def enqueue_shipment(self, shipment):
        self.queue.append(shipment)
        print(f"📦 Shipment {shipment.shipment_id} queued to {shipment.destination}")

    def process_shipment(self):
        if not self.is_empty():
            shipment = self.queue.pop(0)
            shipment.status = "Processed"
            self.processed.append(shipment)
            print(f"🚚 Processing: {shipment.shipment_id} to {shipment.destination}")
            return shipment
        else:
            print(f"⚠️ No shipments to process.")
            return None

    def peek_next_shipment(self):
        if not self.is_empty():
            shipment = self.queue[0]
            print(f"👁️ Next shipment: {shipment.shipment_id} to {shipment.destination}")
            return shipment
        else:
            print(f"⚠️ No shipments in queue.")
            return None

    def is_empty(self):
        return len(self.queue) == 0

    def display_queue(self):
        if self.is_empty():
            print(f"📭 No pending shipments.\n")
        else:
            print(f"\n📭 PENDING SHIPMENTS QUEUE:")
            print("-" * 70)
            for i, shipment in enumerate(self.queue, 1):
                print(f"({i}). ID: {shipment.shipment_id} | Destination: {shipment.destination} | Weight: {shipment.weight}kg")
            print("-" * 70 + "\n")

    def display_processed(self):
        if not self.processed:
            print(f"📭 No processed shipments yet.\n")
        else:
            print(f"\n📭 PROCESSED SHIPMENTS:")
            print("-" * 70)
            for i, shipment in enumerate(self.processed, 1):
                print(f"({i}). ID: {shipment.shipment_id} | Destination: {shipment.destination} | Weight: {shipment.weight}kg")
            print("-" * 70 + "\n")
```

```

# Example Usage
if __name__ == "__main__":
    print("-" * 70)
    print("🚚 SMART LOGISTICS SHIPMENT PROCESSING SYSTEM")
    print("-" * 70 + "\n")

    system = ShipmentQueue()

    # Create and add shipments
    print("📦 Adding shipments to queue:\n")
    s1 = Shipment("SHP-001", "New York", 50)
    s2 = Shipment("SHP-002", "Los Angeles", 75)
    s3 = Shipment("SHP-003", "Chicago", 60)
    s4 = Shipment("SHP-004", "Houston", 45)
    s5 = Shipment("SHP-005", "Miami", 55)

    system.enqueue_shipment(s1)
    system.enqueue_shipment(s2)
    system.enqueue_shipment(s3)
    system.enqueue_shipment(s4)
    system.enqueue_shipment(s5)

    # Display pending queue
    system.display_queue()

    # Peek next shipment
    print("🔍 Checking next shipment:\n")
    system.peek_next_shipment()

    # Process shipments
    print("\n🔄 PROCESSING SHIPMENTS:\n")
    system.process_shipment()
    system.process_shipment()
    system.process_shipment()

    # Display updated queue
    system.display_queue()

    # Display processed shipments
    system.display_processed()

    # Process remaining shipments
    print("Processing remaining shipments:\n")
    system.process_shipment()
    system.process_shipment()

    system.display_queue()
    system.display_processed()

```

OUTPUT:

```

PS C:\Users\chris\OneDrive\Desktop>all git pdfs\LAB_11.2 & c:\python314\python.exe "c:\Users\chris\OneDrive\Desktop\sl\all git pdfs\LAB_11.2\calculator.py"

=====
🚚 SMART LOGISTICS SHIPMENT PROCESSING SYSTEM
=====

📦 Adding shipments to queue:

📦 Shipment SHP-001 queued to New York
📦 Shipment SHP-002 queued to Los Angeles
📦 Shipment SHP-003 queued to Chicago
📦 Shipment SHP-004 queued to Houston
📦 Shipment SHP-005 queued to Miami

📋 PENDING SHIPMENTS QUEUE:
-----
1. ID: SHP-001 | Destination: New York | Weight: 50kg
2. ID: SHP-002 | Destination: Los Angeles | Weight: 75kg
3. ID: SHP-003 | Destination: Chicago | Weight: 60kg
4. ID: SHP-004 | Destination: Houston | Weight: 45kg
5. ID: SHP-005 | Destination: Miami | Weight: 55kg
-----

🔍 Checking next shipment:

📋 Next shipment: SHP-001 to New York

🔄 PROCESSING SHIPMENTS:

📦 Processing: SHP-001 to New York
📦 Processing: SHP-002 to Los Angeles
📦 Processing: SHP-003 to Chicago

📋 PENDING SHIPMENTS QUEUE:
-----
1. ID: SHP-004 | Destination: Houston | Weight: 45kg
2. ID: SHP-005 | Destination: Miami | Weight: 55kg
-----

📋 PROCESSED SHIPMENTS:
-----
1. ID: SHP-001 | Destination: New York | Weight: 50kg
2. ID: SHP-002 | Destination: Los Angeles | Weight: 75kg
3. ID: SHP-003 | Destination: Chicago | Weight: 60kg
-----

Processing remaining shipments:

📦 Processing: SHP-004 to Houston
📦 Processing: SHP-005 to Miami
📦 No pending shipments.

```

```
-----
Processing remaining shipments:

🟡 Processing: SHP-004 to Houston
🟡 Processing: SHP-005 to Miami
🟢 No pending shipments.

🟡 Processing: SHP-004 to Houston
🟡 Processing: SHP-005 to Miami
🟢 No pending shipments.

🟡 Processing: SHP-005 to Miami
🟢 No pending shipments.

🟢 No pending shipments.

✅ PROCESSED SHIPMENTS:
-----
1. ID: SHP-001 | Destination: New York | Weight: 50kg
2. ID: SHP-002 | Destination: Los Angeles | Weight: 75kg
3. ID: SHP-003 | Destination: Chicago | Weight: 60kg
4. ID: SHP-004 | Destination: Houston | Weight: 45kg
5. ID: SHP-005 | Destination: Miami | Weight: 55kg
-----
```

Task Description #14: Smart Banking Management System – Data-Structure Challenge

A bank plans to develop a Smart Banking System that includes:

1. Customer Service Token System – Customers are served in order of arrival.
2. Loan Approval Priority Handling – High-value or urgent loans processed first.
3. Account Information Lookup – Fast retrieval of customer details using Account Number.
4. Transaction History Management – Maintain chronological transaction records.
5. ATM Network Connectivity – Represent connections between ATMs across cities.

Student Task:

- For each feature, select the most appropriate data structure from the list below:
 - o Stack
 - o Queue
 - o Priority Queue
 - o Linked List
 - o Binary Search Tree (BST)
 - o Graph
 - o Hash Table
 - o Deque
- Justify your choice in 2–3 sentences per feature.
- Implement one selected feature as a working Python program

with AI-assisted code generation.

Expected Output:

- A table mapping feature → chosen data structure → justification.
- A functional Python program implementing the chosen feature

with comments and docstrings

```
1212 """
1213 FEATURE MAPPING TABLE:
1214 =====
1215 1. Customer Service Token System → Queue
1216     Justification: Customers must be served in FIFO order based on token arrival time.
1217     Queue ensures fair service and maintains chronological order without favoritism.
1218
1219 2. Loan Approval Priority Handling → Priority Queue
1220     Justification: High-value or urgent loans need expedited processing regardless of arrival time.
1221     Priority queue automatically prioritizes loans by importance, processing critical applications first.
1222
1223
1224
1225 Account Information Lookup → Hash Table
1226     Justification: Provides O(1) average lookup time using Account Number as key.
1227     Enables instant retrieval of customer details, balance, and account status.
1228
1229 4. Transaction History Management → Linked List
1230     Justification: Maintains chronological order of transactions with easy insertion/deletion.
1231     Linked list efficiently stores transaction records without requiring contiguous memory.
1232
1233 5. ATM Network Connectivity → Graph
1234     Justification: ATMs are nodes and connections between cities are edges in the network.
1235     Graph enables pathfinding and connectivity analysis for ATM availability across regions.
1236
1237
1238 # IMPLEMENTATION: Customer Service Token System using Queue
1239
1240 class Token:
1241     """Represents a customer service token."""
1242
1243     def __init__(self, token_id, customer_name, service_type):
1244         self.token_id = token_id
1245         self.customer_name = customer_name
1246         self.service_type = service_type
1247         self.status = "Waiting"
1248
1249     def __repr__(self):
1250         return f"Token({self.token_id}, {self.customer_name}, {self.service_type})"
1251
1252 class BankServiceQueue:
1253     """Manages customer service tokens in FIFO order."""
1254
1255     def __init__(self):
1256         self.queue = []
1257         self.served_tokens = []
1258
1259     def issue_token(self, token_id, customer_name, service_type):
1260         """Issue a new service token to a customer."""
1261         token = Token(token_id, customer_name, service_type)
1262         self.queue.append(token)
1263         print(f"✅ Token #{token_id} issued to {customer_name} for {service_type}")
1264
1265     def call_next_customer(self):
1266         """Call the next customer in queue."""
1267         if not self.is_empty():
1268             token = self.queue.pop(0)
1269             token.status = "Served"
1270             self.served_tokens.append(token)
1271             print(f"📞 Calling: Token #{token.token_id} - {token.customer_name} ({token.service_type})")
1272             return token
1273         else:
1274             print("⚠️ No customers in queue.")
1275             return None
1276
1277
```



```

1274         return None
1275
1276     def peek_next_customer(self):
1277         """Check who is the next customer without removing from queue."""
1278         if not self.is_empty():
1279             token = self.queue[0]
1280             print(f"👤 Next customer: Token #{token.token_id} - {token.customer_name}")
1281             return token
1282         else:
1283             print("⚠️ Queue is empty.")
1284             return None
1285
1286     def is_empty(self):
1287         """Check if queue is empty."""
1288         return len(self.queue) == 0
1289
1290     def display_waiting_queue(self):
1291         """Display all customers waiting in queue."""
1292         if self.is_empty():
1293             print("👥 No customers waiting.\n")
1294         else:
1295             print("\n👥 CUSTOMERS WAITING IN QUEUE:")
1296             print("-" * 70)
1297             for i, token in enumerate(self.queue, 1):
1298                 print(f"{i}. Token #{token.token_id} | {token.customer_name} | {token.service_type}")
1299             print("-" * 70 + "\n")
1300
1301     def display_served_customers(self):
1302         """Display all served customers."""
1303         if not self.served_tokens:
1304             print("👥 No customers served yet.\n")
1305         else:
1306             print("\n✅ SERVED CUSTOMERS:")
1307             print("-" * 70)
1308             for i, token in enumerate(self.served_tokens, 1):
1309                 print(f"{i}. Token #{token.token_id} | {token.customer_name} | {token.service_type}")
1310             print("-" * 70 + "\n")
1311
1312
1313 # Example Usage
1314 if __name__ == "__main__":
1315     print("-" * 70)
1316     print("🏦 SMART BANKING CUSTOMER SERVICE SYSTEM")
1317     print("-" * 70 + "\n")
1318
1319     bank = BankServiceQueue()
1320
1321     # Issue tokens to customers
1322     print("📄 Issuing service tokens:\n")
1323     bank.issue_token(101, "Alice Johnson", "Deposit")
1324     bank.issue_token(102, "Bob Smith", "Withdrawal")
1325     bank.issue_token(103, "Carol White", "Loan Application")
1326     bank.issue_token(104, "David Brown", "Account Opening")
1327     bank.issue_token(105, "Eve Davis", "Balance Inquiry")
1328
1329     # Display waiting queue
1330     bank.display_waiting_queue()
1331
1332     # Check next customer
1333     print("👤 Checking next customer:\n")
1334     bank.peek_next_customer()
1335
1336     # Serve customers
1337     print("\n🔄 SERVING CUSTOMERS:\n")
1338     bank.call_next_customer()
1339     bank.call_next_customer()
1340     bank.call_next_customer()
1341
1342     # Display updated queue
1343     bank.display_waiting_queue()
1344
1345     # Display served customers
1346     bank.display_served_customers()
1347
1348     # Serve remaining customers
1349     print("🔄 Serving remaining customers:\n")
1350     bank.call_next_customer()
1351     bank.call_next_customer()
1352
1353     bank.display_waiting_queue()
1354     bank.display_served_customers()
1355

```


OUTPUT:

```
🟡 Issuing service tokens:

✅ Token #101 issued to Alice Johnson for Deposit
✅ Token #102 issued to Bob Smith for Withdrawal
✅ Token #103 issued to Carol White for Loan Application
✅ Token #104 issued to David Brown for Account Opening
✅ Token #105 issued to Eve Davis for Balance Inquiry
```

```
👤 CUSTOMERS WAITING IN QUEUE:
```

```
-----
1. Token #101 | Alice Johnson | Deposit
2. Token #102 | Bob Smith | Withdrawal
3. Token #103 | Carol White | Loan Application
4. Token #104 | David Brown | Account Opening
5. Token #105 | Eve Davis | Balance Inquiry
-----
```

```
🔍 Checking next customer:
```

```
📄 Next customer: Token #101 - Alice Johnson
```

```
📞 SERVING CUSTOMERS:
```

```
📞 Calling: Token #101 - Alice Johnson (Deposit)
📞 Calling: Token #102 - Bob Smith (Withdrawal)
📞 Calling: Token #103 - Carol White (Loan Application)
```

```
👤 CUSTOMERS WAITING IN QUEUE:
```

```
-----
1. Token #104 | David Brown | Account Opening
2. Token #105 | Eve Davis | Balance Inquiry
-----
```

```
✅ SERVED CUSTOMERS:
```

```
-----
1. Token #101 | Alice Johnson | Deposit
2. Token #102 | Bob Smith | Withdrawal
3. Token #103 | Carol White | Loan Application
-----
```

```
Serving remaining customers:
```

```
📞 Calling: Token #104 - David Brown (Account Opening)
📞 Calling: Token #105 - Eve Davis (Balance Inquiry)
🔍 No customers waiting.
```

```
✅ SERVED CUSTOMERS:
```

```
-----
1. Token #101 | Alice Johnson | Deposit
2. Token #102 | Bob Smith | Withdrawal
3. Token #103 | Carol White | Loan Application
4. Token #104 | David Brown | Account Opening
5. Token #105 | Eve Davis | Balance Inquiry
-----
```

```
PS C:\Users\arell\Music\aiac> 
```

Task Description #15: Online Learning Management System – DataStructure Selection

An online education platform wants to develop a Learning Management System that handles:

1. Student Login Authentication – Quick validation of user credentials.
2. Assignment Submission Queue – Submissions processed in order of upload.
3. Top Performer Ranking – Rank students based on scores.
4. Course Prerequisite Structure – Represent subject dependencies.
5. Undo Feature in Online Editor – Reverse recent actions.

Student Task:

- Select the most appropriate data structure for each feature from the given list.
- Justify your selection in 2–3 sentences per feature.
- Implement one selected feature in Python using AI-assisted code generation.

Expected Output:

- Feature → Data Structure → Justification table.
- Functional Python program with proper documentation.

CODE:

```

1362 FEATURE MAPPING TABLE:
1363 =====
1364 1. Student Login Authentication → Hash Table
1365 | Justification: Provides O(1) average lookup time using username/email as key.
1366 | Enables instant credential validation and fast user authentication without delays.
1367
1368 2. Assignment Submission Queue → Queue
1369 | Justification: Submissions must be processed in FIFO order based on upload time.
1370 | Queue ensures fair evaluation and maintains chronological order of submissions.
1371
1372 3. Top Performer Ranking → Priority Queue
1373 | Justification: Students ranked by scores with highest performers prioritized.
1374 | Priority queue automatically maintains sorted order for quick access to top performers.
1375
1376 4. Course Prerequisite Structure → Graph
1377 | Justification: Courses are nodes and prerequisites are directed edges between them.
1378 | Graph enables validation of course dependencies and pathfinding for course sequences.
1379
1380 5. Undo Feature in Online Editor → Stack
1381 | Justification: Uses LIFO principle to reverse actions in reverse order of execution.
1382 | Stack efficiently stores action history and allows quick undo operations.
1383 """
1384
1385 # IMPLEMENTATION: Assignment Submission Queue using Queue
1386
1387 class Assignment:
1388
1389     def __init__(self, submission_id, student_name, assignment_name, submission_time, score=None):
1390         self.submission_id = submission_id
1391         self.student_name = student_name
1392         self.assignment_name = assignment_name
1393         self.submission_time = submission_time
1394         self.score = score
1395         self.status = "Pending"
1396
1397     def __repr__(self):
1398         return f"Assignment({self.submission_id}, {self.student_name}, {self.assignment_name})"
1399
1400
1401 class AssignmentSubmissionQueue:
1402
1403     def __init__(self):
1404         self.queue = []
1405         self.evaluated = []
1406

```

```

1407 def submit_assignment(self, submission_id, student_name, assignment_name, submission_time):
1408     assignment = Assignment(submission_id, student_name, assignment_name, submission_time)
1409     self.queue.append(assignment)
1410     print(f"📄 Assignment submitted: {student_name} - {assignment_name} at {submission_time}")
1411
1412 def evaluate_next_assignment(self, score):
1413     if not self.is_empty():
1414         assignment = self.queue.pop(0)
1415         assignment.status = "Evaluated"
1416         assignment.score = score
1417         self.evaluated.append(assignment)
1418         print(f"📄 Evaluated: {assignment.student_name}'s {assignment.assignment_name} (Score: {score}/100)")
1419         return assignment
1420     else:
1421         print("⚠️ No assignments to evaluate.")
1422         return None
1423
1424 def peek_next_assignment(self):
1425     if not self.is_empty():
1426         assignment = self.queue[0]
1427         print(f"📄 Next assignment: {assignment.student_name} - {assignment.assignment_name}")
1428         return assignment
1429     else:
1430         print("⚠️ Queue is empty.")
1431         return None
1432
1433 def is_empty(self):
1434     return len(self.queue) == 0
1435
1436 def display_pending_queue(self):
1437     if self.is_empty():
1438         print("📄 No pending assignments.\n")
1439     else:
1440         print("\n📄 PENDING ASSIGNMENTS QUEUE:")
1441         print("-" * 80)
1442         for i, assignment in enumerate(self.queue, 1):
1443             print(f"{i}. ID: {assignment.submission_id} | {assignment.student_name} | {assignment.assignment_name} | {assignment.submission_time}")
1444         print("-" * 80 + "\n")
1445
1446 def display_evaluated_assignments(self):
1447     if not self.evaluated:
1448         print("📄 No evaluated assignments yet.\n")
1449     else:
1450         print("\n📄 EVALUATED ASSIGNMENTS:")
1451         print("-" * 80)
1452         for i, assignment in enumerate(self.evaluated, 1):
1453             print(f"{i}. ID: {assignment.submission_id} | {assignment.student_name} | {assignment.assignment_name} | Score: {assignment.score}/100")
1454         print("-" * 80 + "\n")

```

```

1456
1457 # Example Usage
1458 if __name__ == "__main__":
1459     print("=" * 80)
1460     print("🚀 ONLINE LEARNING MANAGEMENT SYSTEM - ASSIGNMENT SUBMISSION QUEUE")
1461     print("=" * 80 + "\n")
1462
1463     system = AssignmentSubmissionQueue()
1464
1465     # Submit assignments
1466     print("📁 Submitting assignments:\n")
1467     system.submit_assignment(1001, "Alice Johnson", "Python Basics", "09:15 AM")
1468     system.submit_assignment(1002, "Bob Smith", "Data Structures", "09:30 AM")
1469     system.submit_assignment(1003, "Carol White", "Python Basics", "09:45 AM")
1470     system.submit_assignment(1004, "David Brown", "Web Development", "10:00 AM")
1471     system.submit_assignment(1005, "Eve Davis", "Data Structures", "10:15 AM")
1472
1473     # Display pending queue
1474     system.display_pending_queue()
1475
1476     # Check next assignment
1477     print("🔍 Checking next assignment to evaluate:\n")
1478     system.peak_next_assignment()
1479
1480     # Evaluate assignments
1481     print("\n📊 EVALUATING ASSIGNMENTS:\n")
1482     system.evaluate_next_assignment(92)
1483     system.evaluate_next_assignment(85)
1484     system.evaluate_next_assignment(88)
1485
1486     # Display updated queue
1487     system.display_pending_queue()
1488
1489     # Display evaluated assignments
1490     system.display_evaluated_assignments()
1491
1492     # Evaluate remaining assignments
1493     print("Evaluating remaining assignments:\n")
1494     system.evaluate_next_assignment(95)
1495     system.evaluate_next_assignment(78)
1496
1497     system.display_pending_queue()
1498     system.display_evaluated_assignments()

```

OUTPUT:

=====

📁 ONLINE LEARNING MANAGEMENT SYSTEM - ASSIGNMENT SUBMISSION QUEUE

=====

📁 Submitting assignments:

- ✅ Assignment submitted: Alice Johnson - Python Basics at 09:15 AM
- ✅ Assignment submitted: Bob Smith - Data Structures at 09:30 AM
- ✅ Assignment submitted: Carol White - Python Basics at 09:45 AM
- ✅ Assignment submitted: David Brown - Web Development at 10:00 AM
- ✅ Assignment submitted: Eve Davis - Data Structures at 10:15 AM

📁 PENDING ASSIGNMENTS QUEUE:

-
1. ID: 1001 | Alice Johnson | Python Basics | 09:15 AM
 2. ID: 1002 | Bob Smith | Data Structures | 09:30 AM
 3. ID: 1003 | Carol White | Python Basics | 09:45 AM
 4. ID: 1004 | David Brown | Web Development | 10:00 AM
 5. ID: 1005 | Eve Davis | Data Structures | 10:15 AM
-

🔍 Checking next assignment to evaluate:

📁 Next assignment: Alice Johnson - Python Basics

📁 EVALUATING ASSIGNMENTS:

- 📁 Evaluated: Alice Johnson's Python Basics (Score: 92/100)
- 📁 Evaluated: Bob Smith's Data Structures (Score: 85/100)
- 📁 Evaluated: Carol White's Python Basics (Score: 88/100)

📁 PENDING ASSIGNMENTS QUEUE:

-
1. ID: 1004 | David Brown | Web Development | 10:00 AM
 2. ID: 1005 | Eve Davis | Data Structures | 10:15 AM
-

✅ EVALUATED ASSIGNMENTS:

-
1. ID: 1001 | Alice Johnson | Python Basics | Score: 92/100
 2. ID: 1002 | Bob Smith | Data Structures | Score: 85/100
 3. ID: 1003 | Carol White | Python Basics | Score: 88/100
-

Evaluating remaining assignments:

- 📁 Evaluated: David Brown's Web Development (Score: 95/100)
- 📁 Evaluated: Eve Davis's Data Structures (Score: 78/100)
- 📁 No pending assignments.

✅ EVALUATED ASSIGNMENTS:

-
1. ID: 1001 | Alice Johnson | Python Basics | Score: 92/100
 2. ID: 1002 | Bob Smith | Data Structures | Score: 85/100
 3. ID: 1003 | Carol White | Python Basics | Score: 88/100
 4. ID: 1004 | David Brown | Web Development | Score: 95/100
 5. ID: 1005 | Eve Davis | Data Structures | Score: 78/100
-

PS C:\Users\arell\Music\aiac>

Task Description #16: Smart Airport Management System – DataStructure Challenge

An airport authority plans to implement:

1. Passenger Check-In Queue – Process passengers in order of arrival.
2. Emergency Landing Priority – Aircraft in emergency get priority.
3. Flight Information Lookup – Fast search using Flight ID.
4. Air Route Connectivity – Airports connected via routes.
5. Boarding Process (Last-In First-Out Cabin Storage) – Overhead luggage management simulation.

Student Task:

- Choose suitable data structures from the provided list.
- Provide 2–3 sentence justification per feature.
- Implement one feature in Python with comments and docstrings using AI assistance.

Expected Output:

- Mapping table.
- Working Python implementation.

CODE:

```

1499
1500 """
1501 Smart Airport Management System | Data Structure Challenge
1502
1503 An airport authority plans to implement:
1504 1. Passenger Check-In Queue | Process passengers in order of arrival.
1505 2. Emergency Landing Priority | Aircraft in emergency get priority.
1506 3. Flight Information Lookup | Fast search using Flight ID.
1507 4. Air Route Connectivity | Airports connected via routes.
1508 5. Boarding Process (Last-In First-Out Cabin Storage) | Overhead luggage management simulation.
1509
1510 Student Task:
1511 • Choose suitable data structures from the provided list.
1512 • Provide 2-3 sentence justification per feature.
1513 • Implement one feature in Python with comments and docstrings using AI assistance.
1514
1515 Expected Output:
1516 • Mapping table.
1517 • Working Python implementation.
1518 """
1519
1520 # FEATURE MAPPING TABLE
1521 FEATURE_MAPPING = [
1522     (
1523         "Passenger Check-In Queue",
1524         "Queue",
1525         "Passengers must be processed in FIFO order based on arrival time. "
1526         "Queue ensures fair service and maintains chronological processing without delays."
1527     ),
1528     (
1529         "Emergency Landing Priority",
1530         "Priority Queue",
1531         "Aircraft in emergency situations require immediate landing priority. "
1532         "Priority queue sorts by urgency level, ensuring critical aircraft land first."
1533     ),
1534     (
1535         "Flight Information Lookup",
1536         "Hash Table",
1537         "Provides O(1) average lookup time using Flight ID as key. "
1538         "Enables instant retrieval of flight details like schedule, gate, and status."
1539     ),
1540     (
1541         "Air Route Connectivity",
1542         "Graph",
1543         "Airports are nodes and flight routes are edges connecting them. "
1544         "Graph structure enables pathfinding and connectivity analysis for route planning."
1545     ),
1546     (
1547         "Boarding Process (Cabin Storage)",

```



```

1555 def print_feature_mapping_table():
1556     """Display the feature mapping table with justifications."""
1557     print("\n" + "=" * 130)
1558     print("SMART AIRPORT MANAGEMENT SYSTEM - FEATURE MAPPING TABLE")
1559     print("=" * 130)
1560     print(f"{'Feature':<35} {'Data Structure':<18} {'Justification':<75}")
1561     print("-" * 130)
1562     for feature, structure, justification in FEATURE_MAPPING:
1563         print(f"{'feature':<35} {'structure':<18} {'justification':<75}")
1564     print("=" * 130 + "\n")
1565
1566
1567 # IMPLEMENTATION: Passenger Check-In Queue using Queue Data Structure
1568
1569 class Passenger:
1570     """Represents a passenger with check-in details."""
1571
1572     def __init__(self, passenger_id, name, flight_id, arrival_time):
1573         """
1574         Initialize a passenger object.
1575
1576         Args:
1577             passenger_id: Unique identifier for the passenger
1578             name: Name of the passenger
1579             flight_id: Flight ID for the passenger's booking
1580             arrival_time: Time of arrival at check-in counter
1581         """
1582         self.passenger_id = passenger_id
1583         self.name = name
1584         self.flight_id = flight_id
1585         self.arrival_time = arrival_time
1586         self.status = "Waiting"
1587
1588     def __repr__(self):
1589         return f"Passenger({self.passenger_id}, {self.name}, Flight: {self.flight_id}, Time: {self.arrival_time})"
1590
1591
1592 class PassengerCheckInQueue:
1593     """Manages passenger check-in queue using FIFO principle."""
1594
1595     def __init__(self):
1596         """Initialize the check-in queue and tracking lists."""
1597         self.queue = []
1598         self.checked_in = []
1599
1600     def add_passenger(self, passenger_id, name, flight_id, arrival_time):
1601         """
1602         Add a passenger to the check-in queue.
1603
1604         Args:
1605             passenger_id: Unique passenger identifier

```

```

1605         passenger_id: Unique passenger identifier
1606         name: Passenger name
1607         flight_id: Flight ID
1608         arrival_time: Arrival time at counter
1609     """
1610     passenger = Passenger(passenger_id, name, flight_id, arrival_time)
1611     self.queue.append(passenger)
1612     print(f"✅ Passenger added to queue: {name} (ID: {passenger_id}) - Flight {flight_id}")
1613
1614     def process_check_in(self):
1615         """
1616         Process the next passenger in queue (FIFO).
1617
1618         Returns:
1619             Passenger object if processed, None if queue is empty
1620         """
1621         if not self.is_empty():
1622             passenger = self.queue.pop(0)
1623             passenger.status = "Checked-In"
1624             self.checked_in.append(passenger)
1625             print(f"✅ Checked-In: {passenger.name} (Flight: {passenger.flight_id})")
1626             return passenger
1627         else:
1628             print("⚠️ No passengers in queue.")
1629             return None
1630
1631     def peek_next_passenger(self):
1632         """
1633         View the next passenger without removing from queue.
1634
1635         Returns:
1636             Next passenger object if available, None if queue is empty
1637         """
1638         if not self.is_empty():
1639             passenger = self.queue[0]
1640             print(f"👁️ Next passenger: {passenger.name} (Flight: {passenger.flight_id})")
1641             return passenger
1642         else:
1643             print("⚠️ Queue is empty.")
1644             return None
1645
1646     def is_empty(self):
1647         """Check if the queue is empty."""
1648         return len(self.queue) == 0
1649

```

```

1592 class PassengerCheckInQueue:
1593     def display_waiting_passengers(self):
1594         """Display all passengers currently waiting in the queue."""
1595         if self.is_empty():
1596             print("❌ No passengers waiting.\n")
1597         else:
1598             print("\n📋 PASSENGER CHECK-IN QUEUE (FIFO):")
1599             print("-" * 100)
1600             for i, passenger in enumerate(self.queue, 1):
1601                 print(f"{i}. ID: {passenger.passenger_id:<6} | Name: {passenger.name:<20} | Flight: {passenger.flight_id:<8} | Arrival: {passenger.arrival_time}")
1602             print("-" * 100 + "\n")
1603
1604     def display_checked_in_passengers(self):
1605         """Display all passengers who have been checked in."""
1606         if not self.checked_in:
1607             print("❌ No passengers checked-in yet.\n")
1608         else:
1609             print("\n✅ CHECKED-IN PASSENGERS:")
1610             print("-" * 100)
1611             for i, passenger in enumerate(self.checked_in, 1):
1612                 print(f"{i}. ID: {passenger.passenger_id:<6} | Name: {passenger.name:<20} | Flight: {passenger.flight_id:<8} | Status: {passenger.status}")
1613             print("-" * 100 + "\n")
1614
1615 # Example Usage
1616 if __name__ == "__main__":
1617     # Display feature mapping table
1618     print_feature_mapping_table()
1619
1620     print("-" * 100)
1621     print("🏢 SMART AIRPORT MANAGEMENT SYSTEM - PASSENGER CHECK-IN QUEUE")
1622     print("-" * 100 + "\n")
1623
1624     # Create check-in system
1625     system = PassengerCheckInQueue()
1626
1627     # Add passengers to queue
1628     print("➕ Adding passengers to check-in queue:\n")
1629     system.add_passenger(101, "Alice Johnson", "AA-101", "08:00 AM")
1630     system.add_passenger(102, "Bob Smith", "UA-205", "08:05 AM")
1631     system.add_passenger(103, "Carol White", "DL-310", "08:10 AM")
1632     system.add_passenger(104, "David Brown", "AA-101", "08:15 AM")
1633     system.add_passenger(105, "Eve Davis", "UA-205", "08:20 AM")
1634     system.add_passenger(106, "Frank Miller", "SW-415", "08:25 AM")
1635
1636     # Display waiting passengers
1637     system.display_waiting_passengers()
1638
1639     # Peek at next passenger
1640     print("🔍 Checking next passenger:\n")

```

```

1697
1698     # Display waiting passengers
1699     system.display_waiting_passengers()
1700
1701     # Peek at next passenger
1702     print("🔍 Checking next passenger:\n")
1703     system.peak_next_passenger()
1704     print(f"\n📋 Queue Length: {system.get_queue_length()} passengers waiting\n")
1705
1706     # Process check-ins
1707     print("🔄 PROCESSING CHECK-INS:\n")
1708     system.process_check_in()
1709     system.process_check_in()
1710     system.process_check_in()
1711
1712     # Display updated queue
1713     system.display_waiting_passengers()
1714     print(f"📋 Remaining Queue Length: {system.get_queue_length()} passengers\n")
1715
1716     # Display checked-in passengers
1717     system.display_checked_in_passengers()
1718
1719     # Continue processing
1720     print("Continuing to process remaining passengers:\n")
1721     system.process_check_in()
1722     system.process_check_in()
1723     system.process_check_in()
1724
1725     # Final displays
1726     system.display_waiting_passengers()
1727     system.display_checked_in_passengers()
1728
1729     print("🏁 All passengers have been processed!")

```

OUTPUT:

```
=====
SMART AIRPORT MANAGEMENT SYSTEM - FEATURE MAPPING TABLE
=====
Feature                               Data Structure    Justification
-----
Passenger Check-In Queue              Queue             Passengers must be processed in FIFO order based on arrival time. Queue ensures fair service and maintains chronological processing without delays.
Emergency Landing Priority              Priority Queue     Aircraft in emergency situations require immediate landing priority. Priority queue sorts by urgency level, ensuring critical aircraft land first.
Flight Information Lookup              Hash Table        Provides O(1) average lookup time using Flight ID as key. Enables instant retrieval of flight details like schedule, gate, and status.
Air Route Connectivity                 Graph             Airports are nodes and flight routes are edges connecting them. Graph structure enables pathfinding and connectivity analysis for route planning.
Boarding Process (Cabin Storage)       Stack             Uses LIFO principle for overhead luggage management. Stack efficiently handles last-loaded items being retrieved first during deplaning.
=====

❏ SMART AIRPORT MANAGEMENT SYSTEM - PASSENGER CHECK-IN QUEUE
=====

❏ Adding passengers to check-in queue:

✅ Passenger added to queue: Alice Johnson (ID: 101) - Flight AA-101
✅ Passenger added to queue: Bob Smith (ID: 102) - Flight UA-205
✅ Passenger added to queue: Carol White (ID: 103) - Flight DL-310
✅ Passenger added to queue: David Brown (ID: 104) - Flight AA-101
✅ Passenger added to queue: Eve Davis (ID: 105) - Flight UA-205
✅ Passenger added to queue: Frank Miller (ID: 106) - Flight SW-415

✅ PASSENGER CHECK-IN QUEUE (FIFO):
-----
1. ID: 101 | Name: Alice Johnson | Flight: AA-101 | Arrival: 08:00 AM
2. ID: 102 | Name: Bob Smith | Flight: UA-205 | Arrival: 08:05 AM
3. ID: 103 | Name: Carol White | Flight: DL-310 | Arrival: 08:10 AM
4. ID: 104 | Name: David Brown | Flight: AA-101 | Arrival: 08:15 AM
5. ID: 105 | Name: Eve Davis | Flight: UA-205 | Arrival: 08:20 AM
6. ID: 106 | Name: Frank Miller | Flight: SW-415 | Arrival: 08:25 AM
-----

🔍 Checking next passenger:

➡ Next passenger: Alice Johnson (Flight: AA-101)

📊 Queue Length: 6 passengers waiting

🌟 PROCESSING CHECK-INS:

✅ Checked-In: Alice Johnson (Flight: AA-101)
✅ Checked-In: Bob Smith (Flight: UA-205)
✅ Checked-In: Carol White (Flight: DL-310)

✅ PASSENGER CHECK-IN QUEUE (FIFO):
-----
1. ID: 104 | Name: David Brown | Flight: AA-101 | Arrival: 08:15 AM
2. ID: 105 | Name: Eve Davis | Flight: UA-205 | Arrival: 08:20 AM
3. ID: 106 | Name: Frank Miller | Flight: SW-415 | Arrival: 08:25 AM
-----

📊 Remaining Queue Length: 3 passengers
```

```
✅ CHECKED-IN PASSENGERS:
-----
1. ID: 101 | Name: Alice Johnson | Flight: AA-101 | Status: Checked-In
2. ID: 102 | Name: Bob Smith | Flight: UA-205 | Status: Checked-In
3. ID: 103 | Name: Carol White | Flight: DL-310 | Status: Checked-In
-----

Continuing to process remaining passengers:

✅ Checked-In: David Brown (Flight: AA-101)
✅ Checked-In: Eve Davis (Flight: UA-205)
✅ Checked-In: Frank Miller (Flight: SW-415)
🔍 No passengers waiting.

✅ CHECKED-IN PASSENGERS:
-----
1. ID: 101 | Name: Alice Johnson | Flight: AA-101 | Status: Checked-In
2. ID: 102 | Name: Bob Smith | Flight: UA-205 | Status: Checked-In
3. ID: 103 | Name: Carol White | Flight: DL-310 | Status: Checked-In
4. ID: 104 | Name: David Brown | Flight: AA-101 | Status: Checked-In
5. ID: 105 | Name: Eve Davis | Flight: UA-205 | Status: Checked-In
6. ID: 106 | Name: Frank Miller | Flight: SW-415 | Status: Checked-In
-----

🏁 All passengers have been processed!
PS C:\Users\arell\Music\aiac> []
```

Task Description #17: Smart Social Media Platform – Data Structure

Selection

A social media company wants to develop a system that includes:

1. News Feed Generation – Display posts chronologically.
2. Trending Hashtags Tracker – Rank hashtags by usage frequency.
3. User Profile Search – Quick lookup using username.
4. Friend Network Representation – Represent user connections.
5. Message Undo Feature – Allow last sent message to be withdrawn.

Student Task:

- Select appropriate data structures from the list.
- Justify your choices clearly.
- Implement one selected feature as a Python program using AI-assisted code generation.

Expected Output:

- Table mapping features to data structures with justification.
- Working Python code with docstrings and comments.

CODE:

```
1713 FEATURE MAPPING TABLE:
1714 =====
1715 1. News Feed Generation → Linked List
1716     Justification: Maintains chronological order of posts with easy insertion at the beginning.
1717     Linked list efficiently handles dynamic post addition without requiring contiguous memory.
1718
1719 2. Trending Hashtags Tracker → Priority Queue
1720     Justification: Hashtags ranked by usage frequency with most popular appearing first.
1721     Priority queue automatically maintains sorted order for quick access to trending topics.
1722
1723 3. User Profile Search → Hash Table
1724     Justification: Provides O(1) average lookup time using username as key.
1725     Enables instant retrieval of user profiles and account information.
1726
1727 4. Friend Network Representation → Graph
1728     Justification: Users are nodes and friend connections are edges in the network.
1729     Graph structure enables pathfinding and connectivity analysis for friend recommendations.
1730
1731 5. Message Undo Feature → Stack
1732     Justification: Uses LIFO principle to reverse messages in reverse order of sending.
1733     Stack efficiently stores message history and allows quick undo operations.
1734 """
1735
1736 # IMPLEMENTATION: Trending Hashtags Tracker using Priority Queue
1737
1738
1739
1740 class Hashtag:
1741     """Represents a hashtag with its usage count."""
1742
1743     def __init__(self, tag_name, usage_count=1):
1744         self.tag_name = tag_name
1745         self.usage_count = usage_count
1746
1747     def __lt__(self, other):
1748         """Compare hashtags by usage count (higher count = higher priority)."""
1749         return self.usage_count > other.usage_count
1750
1751     def __repr__(self):
1752         return f"#{self.tag_name} ({self.usage_count})"
1753
1754
1755 class TrendingHashtagTracker:
1756     """
1757     Manages trending hashtags using a Priority Queue (min-heap).
1758     Higher usage count = higher priority = more trending.
1759     """
1760
1761     def __init__(self):
1762         self.heap = []
1763         self.hashtag_dict = {}
1764
```

```

1765 def add_hashtag(self, tag_name):
1766     """
1767     Add a hashtag or increment its usage count.
1768
1769     Args:
1770         tag_name: Name of the hashtag (without #)
1771     """
1772     if tag_name in self.hashtag_dict:
1773         self.hashtag_dict[tag_name] += 1
1774         print(f"🔄 Updated #{tag_name} - Usage count: {self.hashtag_dict[tag_name]}")
1775     else:
1776         self.hashtag_dict[tag_name] = 1
1777         print(f"✅ Added new hashtag #{tag_name}")
1778
1779 def get_trending_hashtags(self, limit=5):
1780     """
1781     Retrieve top trending hashtags.
1782
1783     Args:
1784         limit: Number of top hashtags to return
1785
1786     Returns:
1787         List of top trending hashtags
1788     """
1789     heap = []
1790     for tag_name, count in self.hashtag_dict.items():
1791         heapq.heappush(heap, (-count, tag_name))
1792
1793     trending = []
1794     for _ in range(min(limit, len(heap))):
1795         count, tag_name = heapq.heappop(heap)
1796         trending.append((tag_name, -count))
1797
1798     return trending
1799
1800 def display_top_trending(self, limit=5):
1801     """Display top trending hashtags."""
1802     trending = self.get_trending_hashtags(limit)
1803
1804     if not trending:
1805         print("🚫 No hashtags yet.\n")
1806     else:
1807         print(f"\n🔥 TOP {limit} TRENDING HASHTAGS:")
1808         print("-" * 60)
1809         for i, (tag_name, count) in enumerate(trending, 1):
1810             print(f"{i}. #{tag_name:<20} | Usage: {count:>3} times | 📊 {'█' * (count // 5)}")
1811         print("-" * 60 + "\n")

```

```

1812
1813 def display_all_hashtags(self):
1814     """Display all hashtags sorted by usage count."""
1815     if not self.hashtag_dict:
1816         print("🚫 No hashtags yet.\n")
1817     else:
1818         sorted_tags = sorted(self.hashtag_dict.items(), key=lambda x: x[1], reverse=True)
1819         print("\n📋 ALL HASHTAGS (Sorted by Usage):")
1820         print("-" * 60)
1821         for i, (tag_name, count) in enumerate(sorted_tags, 1):
1822             print(f"{i}. #{tag_name:<20} | Usage: {count:>3} times")
1823         print("-" * 60 + "\n")
1824
1825 def remove_hashtag(self, tag_name):
1826     """Remove a hashtag."""
1827     if tag_name in self.hashtag_dict:
1828         del self.hashtag_dict[tag_name]
1829         print(f"❌ Removed hashtag #{tag_name}")
1830     else:
1831         print(f"⚠️ Hashtag #{tag_name} not found.")
1832
1833

```

```

1833
1834 # Example Usage
1835 if __name__ == "__main__":
1836     print("=" * 60)
1837     print("📊 SMART SOCIAL MEDIA PLATFORM - TRENDING HASHTAGS TRACKER")
1838     print("=" * 60 + "\n")
1839
1840     tracker = TrendingHashtagTracker()
1841
1842     # Add hashtags with multiple uses
1843     print("📄 Adding hashtags from posts:\n")
1844     tracker.add_hashtag("Python")
1845     tracker.add_hashtag("DataStructures")
1846     tracker.add_hashtag("Python")
1847     tracker.add_hashtag("Coding")
1848     tracker.add_hashtag("Python")
1849     tracker.add_hashtag("WebDevelopment")
1850     tracker.add_hashtag("DataStructures")
1851     tracker.add_hashtag("Python")
1852     tracker.add_hashtag("AI")
1853     tracker.add_hashtag("Coding")
1854     tracker.add_hashtag("Python")
1855     tracker.add_hashtag("AI")
1856
1857     # Display all hashtags
1858     tracker.display_all_hashtags()
1859
1860     # Display top trending
1861     print("🔥 Checking trending hashtags:\n")
1862     tracker.display_top_trending(5)
1863
1864     # Add more activity to a hashtag
1865     print("📄 More posts using hashtags:\n")
1866     tracker.add_hashtag("AI")
1867     tracker.add_hashtag("AI")
1868     tracker.add_hashtag("AI")
1869
1870     # Display updated trending
1871     tracker.display_top_trending(5)
1872
1873     # Remove a hashtag
1874     print("🗑️ Removing hashtag:\n")
1875     tracker.remove_hashtag("WebDevelopment")
1876
1877     # Final display
1878     tracker.display_all_hashtags()

```


SMART SOCIAL MEDIA PLATFORM - TRENDING HASHTAGS TRACKER

Adding hashtags from posts:

- ✓ Added new hashtag #Python
- ✓ Added new hashtag #DataStructures
- ✗ Updated #Python - Usage count: 2
- ✓ Added new hashtag #Coding
- ✗ Updated #Python - Usage count: 3
- ✓ Added new hashtag #WebDevelopment
- ✗ Updated #DataStructures - Usage count: 2
- ✗ Updated #Python - Usage count: 4
- ✓ Added new hashtag #AI
- ✗ Updated #Coding - Usage count: 2
- ✗ Updated #Python - Usage count: 5
- ✗ Updated #AI - Usage count: 2

ALL HASHTAGS (Sorted by Usage):

1. #Python	Usage: 5 times
2. #DataStructures	Usage: 2 times
3. #Coding	Usage: 2 times
4. #AI	Usage: 2 times
5. #WebDevelopment	Usage: 1 times

Checking trending hashtags:

TOP 5 TRENDING HASHTAGS:

1. #Python	Usage: 5 times	
2. #AI	Usage: 2 times	
3. #Coding	Usage: 2 times	
4. #DataStructures	Usage: 2 times	
5. #WebDevelopment	Usage: 1 times	

More posts using hashtags:

- ✗ Updated #AI - Usage count: 3
- ✗ Updated #AI - Usage count: 4
- ✗ Updated #AI - Usage count: 5

TOP 5 TRENDING HASHTAGS:

1. #AI	Usage: 5 times	
2. #Python	Usage: 5 times	
3. #Coding	Usage: 2 times	
4. #DataStructures	Usage: 2 times	
5. #WebDevelopment	Usage: 1 times	

Task #18: Smart University Hostel Management

Scenario:

A university wants to build a hostel management system that manages:

1. Room Allotment
2. Medical Emergency Handling
3. Student Record Retrieval
4. Hostel Block Connectivity
5. Visitor Entry Log

Student Task

- For each feature, select the most appropriate data structure from the list below:
 - o Stack
 - o Queue
 - o Priority Queue
 - o Linked List
 - o Binary Search Tree (BST)
 - o Graph
 - o Hash Table
 - o Deque
- Justify your choice in 2–3 sentences per feature.
- Implement one selected feature as a working Python program with AI-assisted code generation.

Expected Output:

- A table mapping feature → chosen data structure → justification.
- A functional Python program implementing the chosen feature with comments and docstrings.

```

1871 """
1872 FEATURE MAPPING TABLE:
1873 =====
1874 1. Room Allotment → Queue
1875     Justification: Students are assigned rooms in the order they apply.
1876     Queue ensures fair allocation and maintains chronological order without bias.
1877
1878 2. Medical Emergency Handling → Priority Queue (implemented without heapq)
1879     Justification: Critical patients need expedited attention regardless of arrival time.
1880     Priority queue sorts by severity level to serve emergencies in order of urgency.
1881
1882 3. Student Record Retrieval → Hash Table
1883     Justification: Provides O(1) average lookup time using roll number as key.
1884     Enables instant retrieval of student details and academic records.
1885
1886 4. Hostel Block Connectivity → Graph
1887     Justification: Blocks are nodes and connections between blocks are edges.
1888     Graph structure enables pathfinding and connectivity analysis for navigation.
1889
1890 5. Visitor Entry Log → Linked List
1891     Justification: Maintains chronological order of visitor entries with easy insertion.
1892     Linked list efficiently handles dynamic logging without requiring contiguous memory.
1893 """
1894
1895 # IMPLEMENTATION: Medical Emergency Handling using Priority Queue (without heapq)
1896
1897 class Patient:
1898
1899     def __init__(self, patient_id, name, severity, arrival_time):
1900         self.patient_id = patient_id
1901         self.name = name
1902         self.severity = severity # 1=Critical, 2=High, 3=Medium, 4=Low
1903         self.arrival_time = arrival_time
1904         self.status = "Waiting"
1905
1906     def __repr__(self):
1907         return f"Patient({self.patient_id}, {self.name}, Severity: {self.severity}, Time: {self.arrival_time})"
1908
1909
1910 class MedicalEmergencyQueue:
1911
1912     def __init__(self):
1913         self.queue = []
1914         self.treated = []
1915
1916     def add_patient(self, patient_id, name, severity, arrival_time):
1917         patient = Patient(patient_id, name, severity, arrival_time)
1918         self.queue.append(patient)
1919         # Sort queue by severity (lower number = higher priority)
1920         self.queue.sort(key=lambda p: p.severity)
1921         severity_map = {1: "🔴 CRITICAL", 2: "🟠 HIGH", 3: "🟡 MEDIUM", 4: "🟢 LOW"}
1922         print(f"📌 Patient Added: {name} (ID: {patient_id}) - {severity_map[severity]}")
1923
1924     def treat_next_patient(self):
1925         if not self.is_empty():
1926             patient = self.queue.pop(0)
1927             patient.status = "Treated"
1928             self.treated.append(patient)
1929             severity_map = {1: "CRITICAL", 2: "HIGH", 3: "MEDIUM", 4: "LOW"}
1930             print(f"🏥 Now Treating: {patient.name} (Severity: {severity_map[patient.severity]})")
1931             return patient
1932         else:

```

```

1934     return None
1935
1936     def peek_next_patient(self):
1937         if not self.is_empty():
1938             patient = self.queue[0]
1939             severity_map = {1: "CRITICAL", 2: "HIGH", 3: "MEDIUM", 4: "LOW"}
1940             print(f"👁️ Next Patient: {patient.name} (Severity: {severity_map[patient.severity]})")
1941             return patient
1942         else:
1943             print("❌ No patients in queue.")
1944             return None
1945
1946     def is_empty(self):
1947         return len(self.queue) == 0
1948
1949     def display_queue(self):
1950         if self.is_empty():
1951             print("🚰 No patients waiting.\n")
1952         else:
1953             print("\n🏥 EMERGENCY QUEUE (Sorted by Priority):")
1954             print("-" * 80)
1955             severity_map = {1: "CRITICAL", 2: "HIGH", 3: "MEDIUM", 4: "LOW"}
1956             for patient in self.queue:
1957                 print(f"ID: {patient.patient_id}<6 | Name: {patient.name:<20} | Severity: {severity_map[patient.severity]:<10} | Time: {patient.arrival_time}")
1958             print("-" * 80 + "\n")
1959
1960     def display_treated_patients(self):
1961         if not self.treated:
1962             print("🚰 No treated patients yet.\n")
1963         else:
1964             print("\n✅ TREATED PATIENTS:")
1965             print("-" * 80)
1966             severity_map = {1: "CRITICAL", 2: "HIGH", 3: "MEDIUM", 4: "LOW"}
1967             for patient in self.treated:
1968                 print(f"ID: {patient.patient_id}<6 | Name: {patient.name:<20} | Severity: {severity_map[patient.severity]:<10}")
1969             print("-" * 80 + "\n")

```

```

1972 # Example Usage
1973 if __name__ == "__main__":
1974     print("=" * 80)
1975     print("🏥 SMART UNIVERSITY HOSTEL MEDICAL EMERGENCY SYSTEM")
1976     print("=" * 80 + "\n")
1977
1978     hospital = MedicalEmergencyQueue()
1979
1980     # Add patients with different severity levels
1981     print("📋 Adding Patients to Emergency Queue:\n")
1982     hospital.add_patient(1, "John Doe", 4, "09:00")
1983     hospital.add_patient(2, "Sarah Smith", 1, "09:05")
1984     hospital.add_patient(3, "Mike Johnson", 3, "09:10")
1985     hospital.add_patient(4, "Emily Brown", 2, "09:15")
1986     hospital.add_patient(5, "David Wilson", 1, "09:20")
1987
1988     # Display queue
1989     hospital.display_queue()
1990
1991     # Peek next patient
1992     print("👁️ Checking Next Patient:\n")
1993     hospital.peek_next_patient()
1994
1995     # Treat patients
1996     print("\n🏥 TREATMENT PROCESS:\n")
1997     hospital.treat_next_patient()
1998     hospital.treat_next_patient()
1999     hospital.treat_next_patient()
2000
2001     # Display remaining queue
2002     hospital.display_queue()
2003
2004     # Display treated patients
2005     hospital.display_treated_patients()
2006
2007     # Treat remaining patients
2008     print("Treating remaining patients:\n")
2009     hospital.treat_next_patient()
2010     hospital.treat_next_patient()
2011
2012     hospital.display_treated_patients()

```

SMART UNIVERSITY HOSTEL MEDICAL EMERGENCY SYSTEM

Adding Patients to Emergency Queue:

- ✓ Patient Added: John Doe (ID: 1) - LOW
- ✓ Patient Added: Sarah Smith (ID: 2) - CRITICAL
- ✓ Patient Added: Mike Johnson (ID: 3) - MEDIUM
- ✓ Patient Added: Emily Brown (ID: 4) - HIGH
- ✓ Patient Added: David Wilson (ID: 5) - CRITICAL

EMERGENCY QUEUE (Sorted by Priority):

ID: 2	Name: Sarah Smith	Severity: CRITICAL	Time: 09:05
ID: 5	Name: David Wilson	Severity: CRITICAL	Time: 09:20
ID: 4	Name: Emily Brown	Severity: HIGH	Time: 09:15
ID: 3	Name: Mike Johnson	Severity: MEDIUM	Time: 09:10
ID: 1	Name: John Doe	Severity: LOW	Time: 09:00

Checking Next Patient:

Next Patient: Sarah Smith (Severity: CRITICAL)

TREATMENT PROCESS:

- Now Treating: Sarah Smith (Severity: CRITICAL)
- Now Treating: David Wilson (Severity: CRITICAL)
- Now Treating: Emily Brown (Severity: HIGH)

EMERGENCY QUEUE (Sorted by Priority):

ID: 3	Name: Mike Johnson	Severity: MEDIUM	Time: 09:10
ID: 1	Name: John Doe	Severity: LOW	Time: 09:00

TREATED PATIENTS:

ID: 2	Name: Sarah Smith	Severity: CRITICAL
ID: 5	Name: David Wilson	Severity: CRITICAL
ID: 4	Name: Emily Brown	Severity: HIGH

Treating remaining patients:

- Now Treating: Mike Johnson (Severity: MEDIUM)
- Now Treating: John Doe (Severity: LOW)

TREATED PATIENTS:

ID: 2	Name: Sarah Smith	Severity: CRITICAL
ID: 5	Name: David Wilson	Severity: CRITICAL
ID: 4	Name: Emily Brown	Severity: HIGH
ID: 3	Name: Mike Johnson	Severity: MEDIUM
ID: 1	Name: John Doe	Severity: LOW

Task #19: Smart Electricity Distribution System

Scenario:

An electricity board plans to develop a system that includes:

1. Complaint Handling
2. Fault Priority Handling
3. Consumer Record Lookup
4. Power Grid Connectivity
5. Meter Reading History Tracking

Student Task

- For each feature, select the most appropriate data structure from the list below:
 - o Stack
 - o Queue
 - o Priority Queue
 - o Linked List
 - o Binary Search Tree (BST)
 - o Graph
 - o Hash Table
 - o Deque
- Justify your choice in 2–3 sentences per feature.
- Implement one selected feature as a working Python program with AI-assisted code generation.

Expected Output:

- A table mapping feature → chosen data structure → justification.
- A functional Python program implementing the chosen feature with comments and docstrings.


```

2024 """
2025 FEATURE MAPPING TABLE:
2026 =====
2027 1. Complaint Handling → Queue
2028     Justification: Complaints must be addressed in FIFO order based on arrival time.
2029     Queue ensures fair service and maintains chronological order without bias.
2030
2031 2. Fault Priority Handling → Priority Queue
2032     Justification: Critical faults need expedited repair regardless of arrival time.
2033     Priority queue sorts by severity level to address urgent faults first.
2034
2035 3. Consumer Record Lookup → Hash Table
2036     Justification: Provides O(1) average lookup time using consumer ID as key.
2037     Enables instant retrieval of consumer details, billing info, and account status.
2038
2039 4. Power Grid Connectivity → Graph
2040     Justification: Substations are nodes and transmission lines are edges in the network.
2041     Graph structure enables pathfinding and connectivity analysis for grid management.
2042
2043 5. Meter Reading History Tracking → Linked List
2044     Justification: Maintains chronological order of meter readings with easy insertion.
2045     Linked list efficiently stores historical data without requiring contiguous memory.
2046 """
2047
2048 # IMPLEMENTATION: Complaint Handling System using Queue
2049
2050 class Complaint:
2051
2052     def __init__(self, complaint_id, consumer_id, consumer_name, issue, reported_time):
2053         self.complaint_id = complaint_id
2054         self.consumer_id = consumer_id
2055         self.consumer_name = consumer_name
2056         self.issue = issue
2057         self.reported_time = reported_time
2058         self.status = "Pending"
2059
2060     def __repr__(self):
2061         return f"Complaint({self.complaint_id}, {self.consumer_name}, {self.issue})"
2062
2063
2064 class ComplaintHandlingQueue:
2065
2066     def __init__(self):
2067         self.queue = []
2068         self.resolved = []
2069
2070     def register_complaint(self, complaint_id, consumer_id, consumer_name, issue, reported_time):
2071         complaint = Complaint(complaint_id, consumer_id, consumer_name, issue, reported_time)
2072         self.queue.append(complaint)
2073         print(f"✅ Complaint registered: ID {complaint_id} - {consumer_name} ({issue})")
2074
2075     def resolve_complaint(self):
2076         if not self.is_empty():
2077             complaint = self.queue.pop(0)
2078             complaint.status = "Resolved"
2079             self.resolved.append(complaint)
2080             print(f"✔ Resolved: Complaint {complaint.complaint_id} - {complaint.consumer_name}")
2081             return complaint
2082         else:
2083             print("⚠ No complaints in queue.")
2084             return None

```



```

2080         return None
2081
2082     def peek_next_complaint(self):
2083         if not self.is_empty():
2084             complaint = self.queue[0]
2085             print(f"🔍 Next complaint: ID {complaint.complaint_id} - {complaint.consumer_name} ({complaint.issue})")
2086             return complaint
2087         else:
2088             print("⚠️ Queue is empty.")
2089             return None
2090
2091     def is_empty(self):
2092         return len(self.queue) == 0
2093
2094     def get_queue_length(self):
2095         return len(self.queue)
2096
2097     def display_pending_complaints(self):
2098         if self.is_empty():
2099             print("📄 No pending complaints.\n")
2100         else:
2101             print("\n⚡ PENDING COMPLAINTS QUEUE (FIFO):")
2102             print("-" * 100)
2103             for i, complaint in enumerate(self.queue, 1):
2104                 print(f"{i}. ID: {complaint.complaint_id}<8 | Consumer: {complaint.consumer_name:<20} | Issue: {complaint.issue:<30} | Time: {complaint.reported_time}")
2105             print("-" * 100 + "\n")
2106
2107     def display_resolved_complaints(self):
2108         if not self.resolved:
2109             print("📄 No resolved complaints yet.\n")
2110         else:
2111             print("\n✅ RESOLVED COMPLAINTS:")
2112             print("-" * 100)
2113             for i, complaint in enumerate(self.resolved, 1):
2114                 print(f"{i}. ID: {complaint.complaint_id}<8 | Consumer: {complaint.consumer_name:<20} | Issue: {complaint.issue:<30}")
2115             print("-" * 100 + "\n")
2116
2117
2118
2119
2120
2121

```

```

2122 # Example Usage
2123 if __name__ == "__main__":
2124     print("=" * 100)
2125     print("⚡ SMART ELECTRICITY DISTRIBUTION SYSTEM - COMPLAINT HANDLING QUEUE")
2126     print("=" * 100 + "\n")
2127
2128     system = ComplaintHandlingQueue()
2129
2130     # Register complaints
2131     print("📝 Registering complaints:\n")
2132     system.register_complaint(1001, 5001, "Raj Kumar", "Power outage", "09:00 AM")
2133     system.register_complaint(1002, 5002, "Priya Singh", "High billing", "09:15 AM")
2134     system.register_complaint(1003, 5003, "Amit Patel", "Meter malfunction", "09:30 AM")
2135     system.register_complaint(1004, 5004, "Sneha Gupta", "Damaged pole", "09:45 AM")
2136     system.register_complaint(1005, 5005, "Vikram Shah", "Connection issue", "10:00 AM")
2137
2138     # Display pending complaints
2139     system.display_pending_complaints()
2140
2141     # Peek at next complaint
2142     print("🔍 Checking next complaint:\n")
2143     system.peak_next_complaint()
2144     print(f"\n📄 Total Pending Complaints: {system.get_queue_length()}\n")
2145
2146     # Resolve complaints
2147     print("\ RESOLVING COMPLAINTS:\n")
2148     system.resolve_complaint()
2149     system.resolve_complaint()
2150     system.resolve_complaint()
2151
2152     # Display updated queue
2153     system.display_pending_complaints()
2154
2155     # Display resolved complaints
2156     system.display_resolved_complaints()
2157
2158     # Resolve remaining complaints
2159     print("Resolving remaining complaints:\n")
2160     system.resolve_complaint()
2161     system.resolve_complaint()
2162
2163     system.display_pending_complaints()
2164     system.display_resolved_complaints()
2165
2166     print("✅ All complaints have been resolved!")

```

OUTPUT:

```
⚡ SMART ELECTRICITY DISTRIBUTION SYSTEM - COMPLAINT HANDLING QUEUE
=====

📌 Registering complaints:

✅ Complaint registered: ID 1001 - Raj Kumar (Power outage)
✅ Complaint registered: ID 1002 - Priya Singh (High billing)
✅ Complaint registered: ID 1003 - Amit Patel (Meter malfunction)
✅ Complaint registered: ID 1004 - Sneha Gupta (Damaged pole)
✅ Complaint registered: ID 1005 - Vikram Shah (Connection issue)

⚡ PENDING COMPLAINTS QUEUE (FIFO):
-----
1. ID: 1001 | Consumer: Raj Kumar | Issue: Power outage | Time: 09:00 AM
2. ID: 1002 | Consumer: Priya Singh | Issue: High billing | Time: 09:15 AM
3. ID: 1003 | Consumer: Amit Patel | Issue: Meter malfunction | Time: 09:30 AM
4. ID: 1004 | Consumer: Sneha Gupta | Issue: Damaged pole | Time: 09:45 AM
5. ID: 1005 | Consumer: Vikram Shah | Issue: Connection issue | Time: 10:00 AM
-----

🔍 Checking next complaint:

📌 Next complaint: ID 1001 - Raj Kumar (Power outage)

📊 Total Pending Complaints: 5

🔧 RESOLVING COMPLAINTS:

✅ Resolved: Complaint 1001 - Raj Kumar
✅ Resolved: Complaint 1002 - Priya Singh
✅ Resolved: Complaint 1003 - Amit Patel

⚡ PENDING COMPLAINTS QUEUE (FIFO):
-----
1. ID: 1004 | Consumer: Sneha Gupta | Issue: Damaged pole | Time: 09:45 AM
2. ID: 1005 | Consumer: Vikram Shah | Issue: Connection issue | Time: 10:00 AM
-----

✅ RESOLVED COMPLAINTS:
-----
1. ID: 1001 | Consumer: Raj Kumar | Issue: Power outage
2. ID: 1002 | Consumer: Priya Singh | Issue: High billing
3. ID: 1003 | Consumer: Amit Patel | Issue: Meter malfunction
-----

Resolving remaining complaints:

✅ Resolved: Complaint 1004 - Sneha Gupta
✅ Resolved: Complaint 1005 - Vikram Shah
📌 No pending complaints.

✅ RESOLVED COMPLAINTS:
-----
1. ID: 1001 | Consumer: Raj Kumar | Issue: Power outage
2. ID: 1002 | Consumer: Priya Singh | Issue: High billing
3. ID: 1003 | Consumer: Amit Patel | Issue: Meter malfunction
4. ID: 1004 | Consumer: Sneha Gupta | Issue: Damaged pole
5. ID: 1005 | Consumer: Vikram Shah | Issue: Connection issue
-----

✅ All complaints have been resolved!
```


